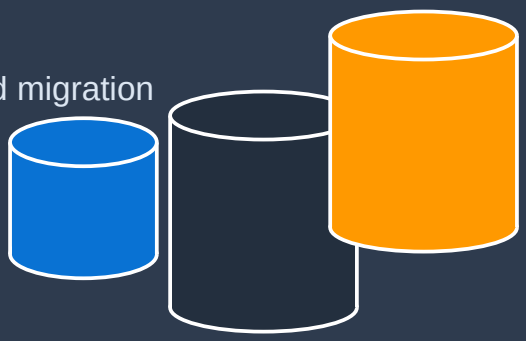
ARCHITECTURE | CONFIGURATION | MULTI-AZ | MIGRATION

AWS RDS for SQL Server

A Practical DBA Guide

Architecture, setup, Multi-AZ and DR, operations, and migration from on-premises to RDS and from RDS to EC2



WHAT YOU WILL LEARN

- AWS basics and RDS architecture
- Step-by-step configuration, security and sizing
- Multi-AZ, read replicas and disaster recovery
- Backup, restore, maintenance and monitoring
- Migration: on-premises to RDS, and RDS to EC2

Prepared October 2026. Versions, limits and features change often; verify exact values in the AWS documentation.

Contents

How to Use This Guide . . . 5

1. AWS Basics for a SQL Server DBA . . . 6

1.1 On-premises terms and their AWS equivalents . . . 6

1.2 Regions, Availability Zones and the VPC . . . 6

1.3 The shared responsibility model . . . 7

1.4 Core AWS services you will meet . . . 8

1.5 Ways to work with AWS . . . 8

2. Amazon RDS for SQL Server: What It Is and How It Is Built . . . 9

2.1 Architecture at a glance . . . 9

2.2 What AWS does and what you do . . . 10

2.3 Editions, versions and licensing . . . 10

2.4 Instance classes . . . 10

2.5 What you cannot do on RDS (and the workarounds) . . . 11

2.6 RDS, RDS Custom or EC2? . . . 11

3. Planning: Sizing, Storage and Cost . . . 13

3.1 Storage types . . . 13

3.2 How to size an instance . . . 13

3.3 tempdb, memory and parallelism . . . 14

3.4 Cost levers . . . 14

4. Configuring RDS for SQL Server Step by Step . . . 15

4.1 Prerequisites checklist . . . 15

4.2 Step 1: Network . . . 15

4.3 Step 2: Parameter group, option group and IAM role . . . 16

4.4 Step 3: Create the DB instance . . . 17

4.5 Step 4: Connect and verify . . . 18

4.6 Windows Authentication (optional) . . . 18

4.7 Step 5: Day-one hardening checklist . . . 19

5. Security . . . 20

5.1 Network security . . . 20

5.2 Authentication and authorization . . . 21

5.3 Encryption . . . 21

5.4 Protecting data inside the database . . . 21

5.5 Auditing and detection . . . 22

5.6 Security checklist . . . 22

6. High Availability, Multi-AZ and Disaster Recovery . . . 23

6.1 Key terms . . . 23

6.2 How Multi-AZ works . . . 23

6.3 What happens during a failover . . . 24

6.4 Configuring Multi-AZ . . . 25

6.5 Making applications failover-ready . . . 25

6.6 Read replicas (Enterprise Edition) . . . 25

6.7 Disaster recovery options . . . 26

6.8 Choosing a design . . . 27

7. Backup, Restore and Maintenance . . . 28

7.1 Automated backups . . . 28

7.2 Manual snapshots and copies . . . 28

7.3 Restoring . . . 29

7.4 Native backup and restore with S3 . . . 29

7.5 Maintenance and upgrades . . . 30

8. Monitoring and Day-to-Day Operations . . . 31

8.1 The monitoring toolbox . . . 31

8.2 Metrics every DBA should watch . . . 31

8.3 Logs and the SQL Server error log . . . 32

8.4 Finding what is slow inside SQL Server . . . 32

8.5 Routine DBA tasks that are still yours . . . 32

8.6 Troubleshooting guide . . . 33

9. Migrating from On-Premises SQL Server to Amazon RDS . . . 34

9.1 Phase 1: Assess . . . 34

9.2 Phase 2: Design the target . . . 35

9.3 Moving the data: bandwidth and transfer options . . . 35

9.4 Path A in detail: native backup + log restore (near-zero downtime) . . . 36

9.5 Path B in detail: AWS DMS . . . 37

9.6 Server-level objects that do not travel with the database . . . 38

9.7 Phase 4: Validate and optimise . . . 38

9.8 Sample cutover runbook . . . 39

9.9 Common pitfalls . . . 39

10. Migrating from Amazon RDS to SQL Server on EC2 . . . 40

10.1 Why move from RDS to EC2? . . . 40

10.2 Designing the EC2 target . . . 40

10.3 What you must now build yourself . . . 42

10.4 Migration methods from RDS 42
10.5 Step-by-step: native backup path 43
10.6 Cutover runbook (RDS to EC2) 44
Appendix A: DBA Cheat Sheet 45
RDS stored procedures commonly used 45
AWS CLI commands commonly used 45
Appendix B: Which Hosting Model and Migration Method? 45
Appendix C: Glossary 46
Appendix D: Where to Learn More 46

How to Use This Guide

This guide is written for a SQL Server DBA who is new to AWS. It explains how Amazon RDS for SQL Server is built, how to configure it, how Multi-AZ and disaster recovery work, how to run it day to day, and how to migrate in both directions: from on-premises to RDS, and from RDS to SQL Server on EC2.

Chapter	You will learn	Read it when you...
1. AWS Basics	Regions, AZs, VPC, IAM, S3, KMS in DBA terms	are new to AWS
2. RDS Architecture	How RDS is built, editions, restrictions, RDS vs Custom vs EC2	must choose a hosting model
3. Planning	Storage types, sizing, tempdb, cost levers	size an instance
4. Configuration	Network, parameter/option groups, create, connect, harden	build an instance
5. Security	Network, identity, encryption, audit, checklist	own security
6. HA and DR	Multi-AZ internals, failover, read replicas, cross-Region DR	design resilience
7. Backup and Maintenance	Automated backups, snapshots, restore, native backup, upgrades	own data protection
8. Monitoring	CloudWatch, Performance Insights, logs, DMVs, troubleshooting	run production
9. On-premises to RDS	Assess, plan, native backup + log restore, DMS, cutover	migrate to AWS
10. RDS to EC2	Why, EC2 design, native backup path, DMS, cutover	move off RDS

A note on accuracy

RDS for SQL Server evolves quickly. In this guide: Multi-AZ is available with Always On availability groups, database mirroring (older builds) or block-level replication (SQL Server 2022 Web), and large volume limits have been raised in 2024-2026. Values are given as planning guidance. Always confirm versions, limits, SLAs and command syntax in the current AWS documentation before building or migrating.

How the boxes work

Blue boxes explain ideas simply, green boxes give good practice, amber boxes warn about traps, and purple boxes summarise what to remember.

1. AWS Basics for a SQL Server DBA

If you know SQL Server on-premises, you already know most of what RDS does. What is new is the vocabulary around it: regions, VPCs, security groups, IAM and S3. This chapter maps those new ideas to things you already understand.

1.1 On-premises terms and their AWS equivalents

On-premises world	AWS equivalent	What a DBA should know
Physical or virtual server	Amazon EC2 instance	RDS runs on EC2 behind the scenes, but you never see the OS
SAN / local disks	Amazon EBS volumes	RDS storage is EBS. Type (gp3/io2), size and IOPS are your choices
Backup share / tape	Amazon S3	Native .bak files, snapshots exports and logs live in S3 buckets
VLAN / network segment	VPC and subnets	RDS must live in a VPC, inside subnets you choose
Firewall rules	Security groups (and NACLs)	A security group on RDS decides who can reach port 1433
Active Directory	AWS Directory Service / IAM	Windows Authentication uses AWS Managed Microsoft AD
Datacentre / DR site	Availability Zone / Region	Multi-AZ = second data centre; cross-Region = DR site
Failover cluster / AG	Multi-AZ deployment	RDS configures Always On or mirroring for you
Perfmon, alerts, SQL Agent alerts	CloudWatch, SNS, Performance Insights	Metrics, alarms and event notifications
sp_configure / server settings	DB parameter group	Settings are changed in parameter groups, not by sp_configure
Optional features, install media	Option group	Features such as native backup or TDE are enabled with options
Service account passwords	Secrets Manager / IAM	Store and rotate the master password safely

1.2 Regions, Availability Zones and the VPC

A **Region** is a geographic area (for example Mumbai). Each Region has several **Availability Zones (AZs)**: separate data centres with independent power and networking, connected by fast links. Your private network inside a Region is a **VPC (Virtual Private Cloud)**, divided into **subnets**; each subnet lives in exactly one AZ. This is the foundation of Multi-AZ.

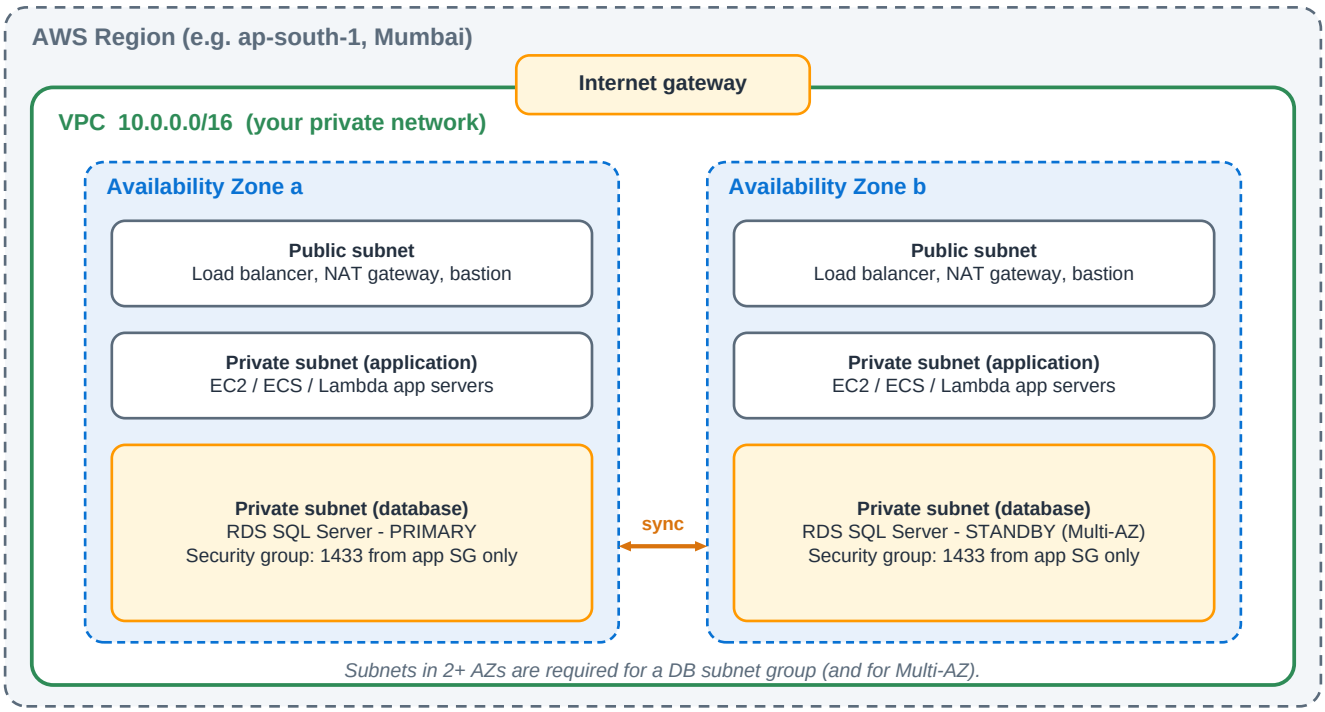


Figure 1.1 - Region, VPC, Availability Zones and subnets for an RDS deployment

- **Public subnet** - has a route to the internet gateway. Put load balancers and NAT gateways here, never the database.
- **Private subnet** - no direct internet route. Application servers and RDS live here.
- **Security group** - a stateful virtual firewall attached to the RDS instance. Allow port 1433 only from the application's security group and trusted corporate ranges.
- **NACL** - an optional stateless filter at subnet level. Most teams leave NACLs at defaults and rely on security groups.

1.3 The shared responsibility model

AWS secures the cloud itself (data centres, hardware, hypervisor). You secure what you put in it. For RDS, AWS takes on far more than for EC2:

Task	On-prem	SQL Server on EC2	RDS Custom	Amazon RDS
Hardware, data centre, hypervisor	You	AWS	AWS	AWS
Operating system install and patching	You	You	Shared (automation + you)	AWS
SQL Server install and patching	You	You	Shared	AWS (minor updates in maintenance window)
Backups and point-in-time restore	You	You	AWS automation	AWS (automated)
High availability setup	You	You	You (RDS Custom has Multi-AZ options for some versions: check docs)	AWS (Multi-AZ checkbox)
Index and query tuning, schema	You	You	You	You
Logins, permissions, data security	You	You	You	You

Task	On-prem	SQL Server on EC2	RDS Custom	Amazon RDS
Network rules (security groups)	You	You	You	You

1.4 Core AWS services you will meet

Service	Purpose	Where it shows up with RDS SQL Server
IAM	Users, roles, policies	Who may create/modify RDS; the IAM role RDS uses to read/write S3 for native backups
KMS	Encryption keys	Encrypts RDS storage, snapshots, backups and native .bak files
S3	Object storage	Native backup files, log-shipping files, export/import data
CloudWatch	Metrics, logs, alarms	CPU, IOPS, free storage, SQL error log and agent log, alarms
CloudTrail	Audit of AWS API calls	Who changed the instance, snapshot or security group
Secrets Manager	Store and rotate secrets	RDS can manage the master user password for you
Systems Manager	Operations tooling	Run scripts on EC2, patch, parameter storage
AWS DMS	Database Migration Service	Full load + ongoing replication for migrations
Direct Connect / Site-to-Site VPN	Private connectivity to on-premises	Move data and connect applications during migration

1.5 Ways to work with AWS

- **AWS Management Console** - web UI. Good for learning and one-off changes.
- **AWS CLI** (`aws rds ...`) and SDKs - scripting and automation.
- **Infrastructure as code** - CloudFormation, CDK or Terraform, so every environment is repeatable.
- **SSMS, sqlcmd, Azure Data Studio alternatives** - connect to the RDS endpoint just like any SQL Server (port 1433, 'host,port' format).

2. Amazon RDS for SQL Server: What It Is and How It Is Built

Amazon RDS for SQL Server is a managed service that runs real Microsoft SQL Server for you. AWS provisions the machine, installs and patches Windows and SQL Server, takes backups, and can keep a synchronous standby copy in another Availability Zone. You keep control of the databases, schema, queries, users and tuning.

2.1 Architecture at a glance

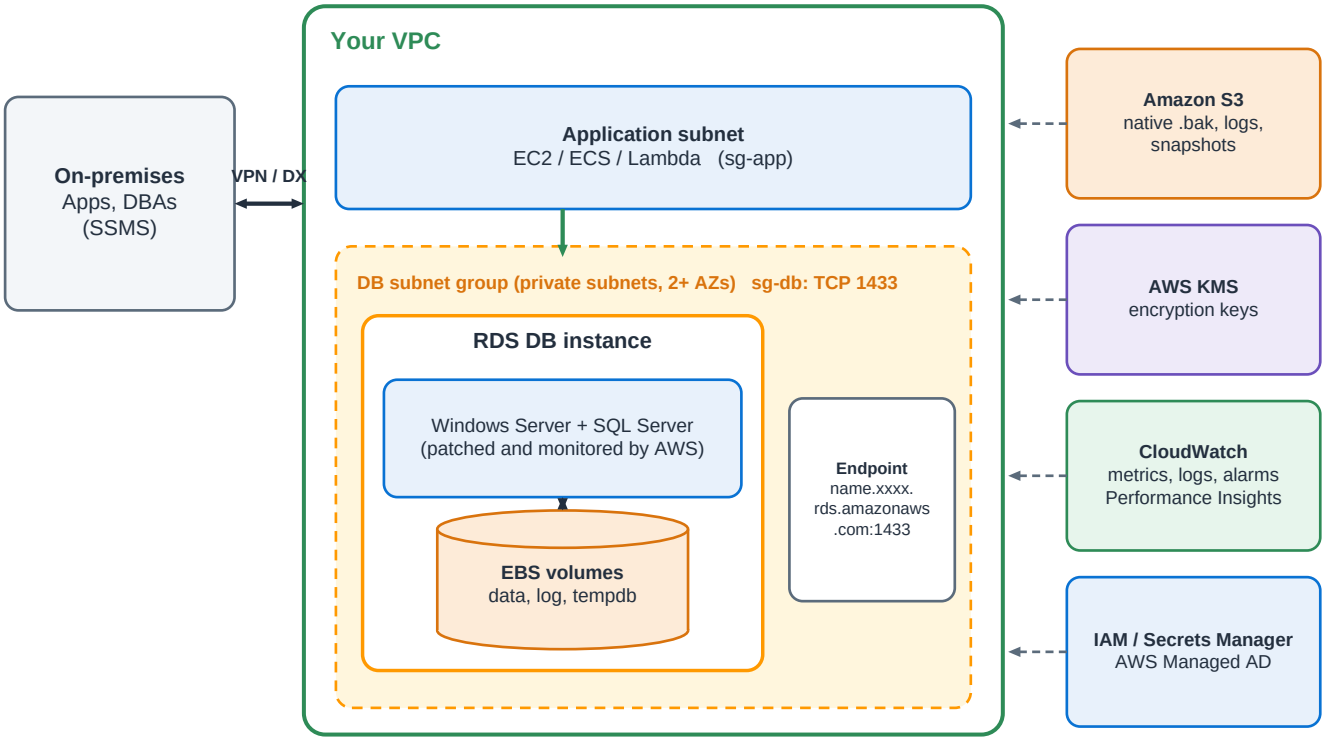


Figure 2.1 - Architecture of an RDS for SQL Server deployment

Key points to notice:

- A **DB instance** is a managed Windows server with SQL Server installed. Data, log and tempdb files live on **EBS volumes**.
- You connect to a stable **DNS endpoint** on port 1433. Never use the underlying IP address; it can change on failover or maintenance.
- The instance sits in a **DB subnet group**: a set of private subnets in at least two AZs that tells RDS where it may place the instance.
- A **security group** is the firewall. Applications and on-premises networks reach the endpoint through your VPC (peering, VPN or Direct Connect).
- **S3, KMS, CloudWatch and IAM** support backups, encryption, monitoring and access control.

2.2 What AWS does and what you do

AWS handles	You handle
Provisioning, hardware, Windows and SQL Server installation	Database design, schemas, stored procedures
OS and SQL Server minor-version patching (maintenance window)	Index maintenance, statistics, DBCC CHECKDB scheduling
Automated backups, transaction log capture, point-in-time restore	Choosing retention, testing restores
Multi-AZ replication and automatic failover	Application retry logic and connection settings
Storage and compute scaling operations	Capacity planning, cost control, choosing instance class
Hardware monitoring, health checks, event notifications	Query performance monitoring, alarms, runbooks
Encryption at rest (with KMS) and TLS endpoint	Logins, roles, least privilege, auditing needs

2.3 Editions, versions and licensing

Edition	Typical use	Multi-AZ	Read replicas	Notes
Express	Dev/test, tiny apps	No	No	Database size limit (about 10 GB); limited CPU and memory
Web	Web hosting workloads	Only SQL Server 2022 (block-level replication)	No	Lower licence cost; fewer features
Standard	Most production workloads	Yes (Always On or mirroring by version)	No	SQL Server Standard edition limits on cores and memory apply
Enterprise	Mission-critical, large	Yes	Yes (Enterprise only)	All features, online index operations, partitioning, etc.

RDS currently offers SQL Server 2016 through 2025 in various minor versions. Older versions are retired on a published schedule, so always check the AWS 'SQL Server versions on Amazon RDS' and version-policy pages before choosing.

- **Licensing** - Amazon RDS for SQL Server uses **licence-included** pricing (the SQL Server licence is part of the hourly price). If you need to bring your own licence and media, look at **RDS Custom for SQL Server** or EC2.
- **Optimize CPUs** - on some instance classes you can reduce the number of active vCPUs while keeping memory and I/O, which lowers licence-included cost for memory- or I/O-bound workloads.
- **Compatibility level** - each database keeps its own compatibility level, so you can upgrade the engine and raise compatibility later.

2.4 Instance classes

Family	Examples	Good for
General purpose (M)	db.m5, db.m6i, db.m7i	Balanced CPU and memory; most OLTP
Memory optimised (R, X)	db.r5, db.r6i, db.r7i, db.x2iedn	Large buffer pool, reporting, memory-hungry workloads
Burstable (T)	db.t3	Dev/test and small, spiky workloads (monitor CPU credits)
Classes with local NVMe ('d')	db.m5d, db.r5d, ...	tempdb on fast local instance store (where supported)

Available classes depend on Region and edition. Check the current list in the console or the RDS documentation.

2.5 What you cannot do on RDS (and the workarounds)

RDS trades some freedom for automation. These are the restrictions DBAs hit most often. Always verify against the current 'Microsoft SQL Server features on Amazon RDS' and 'Common DBA tasks' pages.

Restriction	What it means	Workaround
No RDP or OS access	You cannot log on to the Windows host or install agents/drivers	Use RDS features, CloudWatch, or move to RDS Custom/EC2
No sysadmin role	The master user has broad but limited rights; many server-level commands are blocked	Use RDS stored procedures (rdsadmin / msdb.dbo.rds_*) for admin tasks
No access to the file system	RESTORE/BACKUP to local disk and BULK INSERT from local paths are not available	Use native backup to S3, or the S3 integration option
Cannot restore msdb or master	System databases are managed by RDS	Script out jobs, logins, linked servers and re-create them
Cannot change many server settings with sp_configure	RDS exposes a subset	Use DB parameter groups (max degree of parallelism, cost threshold, etc.)
Limited number of user databases	The limit depends on instance class and Multi-AZ mode	Check current limits; consolidate or use more instances
Cannot rename a database in Multi-AZ	Rename helper is blocked while Multi-AZ is on	Rename in Single-AZ, or use rdsadmin rename procedure where allowed
Cannot stop a Multi-AZ instance	Stop/start works only for Single-AZ	Use snapshots to save cost, or delete and restore
Some features unavailable	e.g. FILESTREAM, PolyBase, Machine Learning Services, Stretch DB (verify list)	Use RDS Custom/EC2 or redesign

2.6 RDS, RDS Custom or EC2?

There are three ways to run SQL Server on AWS. They sit on a spectrum from 'AWS does most of the work' to 'you do everything'.

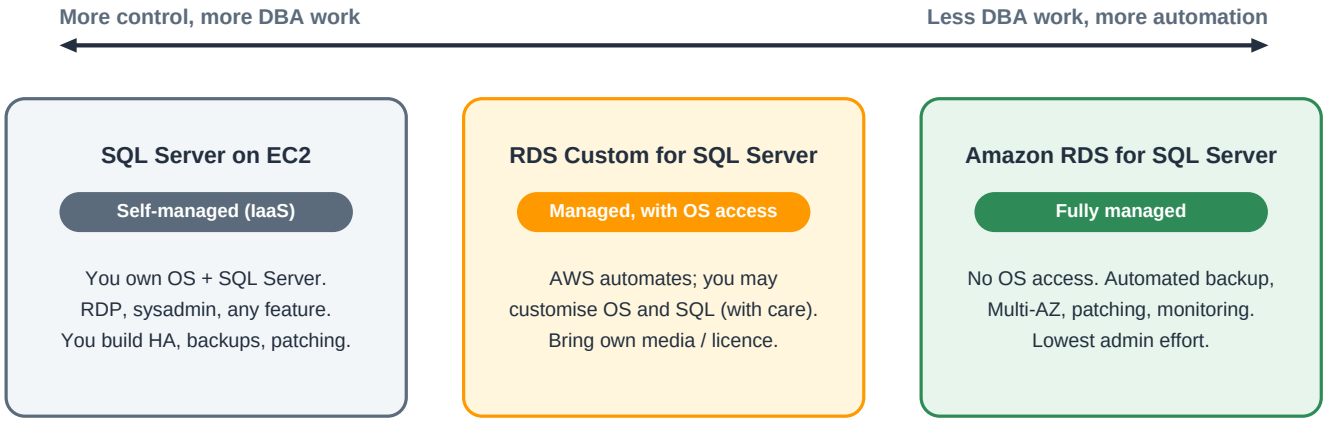


Figure 2.2 - Choosing a hosting model for SQL Server on AWS

Question	Amazon RDS	RDS Custom	SQL Server on EC2
OS access (RDP)	No	Yes	Yes
sysadmin and full server config	No	Yes	Yes
Automated backups / PITR	Yes	Yes	You build (or use AWS Backup / scripts)

Question	Amazon RDS	RDS Custom	SQL Server on EC2
Automatic patching	Yes	Automation with your control	You
Multi-AZ HA	One-click	Limited options; check docs	You design (Always On AG / FCI)
Bring your own licence / media	No (licence included)	Yes	Yes
Third-party agents, SSAS/SSRS on same host	Limited	Yes	Yes
Typical admin effort	Lowest	Medium	Highest

> **Rule of thumb**
> Start with **RDS**. Choose **RDS Custom** only when a specific OS-level or server-level need blocks RDS but you still want AWS automation. Choose **EC2** when you need complete control or an unsupported feature, and accept the extra operational work.

3. Planning: Sizing, Storage and Cost

Most RDS problems come from picking the wrong instance class or storage settings. This chapter shows how to size, with the limits that matter.

3.1 Storage types

RDS storage is Amazon EBS. **Performance is limited by the smaller of two things**: what the volume can deliver and what the instance class can move (instance-level EBS bandwidth and IOPS caps). A huge volume on a tiny instance cannot go faster than the instance allows.

Type	Best for	Key points
gp3 (General Purpose SSD)	Most workloads	IOPS and throughput can be set independently of size. Recent updates raised the per-volume limits significantly (check current numbers).
io2 Block Express (Provisioned IOPS)	Latency-sensitive, high-IOPS OLTP and data warehouses	Sub-millisecond latency, up to 64 TiB and 256,000 IOPS per volume on supported classes. Recommended by AWS for demanding workloads.
io1 (older Provisioned IOPS)	Legacy	Previous generation; prefer io2 Block Express where available.
gp2 (older General Purpose)	Legacy	IOPS tied to size with burst credits; watch the burst balance metric.
Magnetic (standard)	Not recommended	Being retired (deprecation announced for April 2026). Move to gp3 or io2.

- **Size and growth** - SQL Server instances can use multiple storage volumes (additional volumes raise total capacity to 256 TiB in current documentation). Verify the current limits for your edition.
- **Storage autoscaling** - set a maximum storage threshold so RDS grows storage automatically when free space runs low. Storage can be increased but not decreased, and there is a cooldown between modifications.
- **Encryption** - choose at creation time. An unencrypted instance cannot be encrypted in place; you must restore an encrypted copy of a snapshot.

3.2 How to size an instance

Measure the source before choosing. Collect at least one full business cycle (a month if possible) of these counters from the on-premises server:

Measure	Perfmon counter / source	Maps to
CPU	Processor: % Processor Time (peak and 95th percentile)	vCPU count
Memory	SQL Server buffer pool size, Page Life Expectancy, Memory Grants Pending	Instance memory (R-class if large)
IOPS	Disk Reads/sec + Disk Writes/sec (per volume)	Provisioned IOPS and instance EBS cap
Throughput	Disk Read/Write Bytes/sec	Volume and instance throughput
Latency	Avg. Disk sec/Read and sec/Write	Storage type (gp3 vs io2)
Network	Network Interface: Bytes Total/sec	Instance network bandwidth
Data size and growth	sys.master_files, backup history	Allocated storage and autoscaling limit

Measure	Perfmon counter / source	Maps to
tempdb use	tempdb file size, version store	Local NVMe class or larger volumes

Good practice

Add 20-30% headroom to peak numbers, start one size smaller than the first instinct if usage is spiky, and plan a performance test with a production-like workload before cutover. Scaling an RDS instance class is easy but causes a short outage (or a failover in Multi-AZ).

3.3 tempdb, memory and parallelism

- **tempdb** - on supported 'd' classes you can place tempdb on local NVMe instance store for very low latency. Local data is not durable, but tempdb is rebuilt on restart anyway.
- **max server memory** - RDS manages this setting for you. If you must change it, use the DB parameter group.
- **MAXDOP and cost threshold for parallelism** - set through the parameter group. Reset them away from defaults for OLTP workloads (for example MAXDOP 4 or 8, cost threshold 50 as a starting point).

3.4 Cost levers

Lever	How it saves money	Watch out for
Reserved Instances (1 or 3 years)	Large discount on steady production instances	Commitment; match size family
Right-size and re-check	Pay for what the workload really needs	Review CPU and memory trends monthly
Optimize CPUs	Fewer active vCPUs means lower licence-included cost	Only for suitable workloads and classes
Single-AZ for non-production	Multi-AZ roughly doubles instance cost	Never do this for production
Stop dev/test instances	No instance charge while stopped (storage still billed)	Single-AZ only; AWS restarts after a maximum stop period
Edition choice	Standard instead of Enterprise where features allow	Read replicas and some features need Enterprise
Storage tuning	gp3 with just enough IOPS instead of over-provisioned io	Measure before reducing
Snapshot housekeeping	Delete old manual snapshots and unused exports	Keep what compliance requires

4. Configuring RDS for SQL Server Step by Step

This chapter is a hands-on walkthrough: from an empty AWS account to a hardened, production-ready SQL Server instance. Each step shows the idea first and then the console or CLI action.



Figure 4.1 - The five configuration steps

4.1 Prerequisites checklist

Item	Decision to make before you start
Region and AZs	Choose the Region closest to users/data sources and pick a second Region for DR
Edition and version	Standard or Enterprise? Which SQL Server version (check features and support dates)?
Sizing	Instance class, storage type, size, IOPS (see chapter 3)
Availability	Multi-AZ for production; read replicas if reporting or DR needs them (Enterprise)
Network	VPC, two or more private subnets in different AZs, how on-premises will connect
Authentication	SQL logins only, or Windows Authentication through AWS Managed Microsoft AD
Encryption	KMS key (customer-managed key recommended), TDE need, TLS requirement
Collation and time zone	Set at creation: choose carefully (match the source for migrations)
Backup policy	Retention days, backup window, cross-Region copy, long-term archive
Maintenance window	A low-traffic window for patching and Multi-AZ failovers

4.2 Step 1: Network

Create (or reuse) a VPC with at least two private subnets in different Availability Zones. Then create a **DB subnet group** that lists those subnets and a **security group** that allows TCP 1433 only from approved sources.

```
# DB subnet group across two AZs
aws rds create-db-subnet-group --db-subnet-group-name sql-subnets \
  --db-subnet-group-description "Private subnets for SQL Server" \
  --subnet-ids subnet-0aaa1111 subnet-0bbb2222

# Security group for the database; allow 1433 only from the application's security group
aws ec2 create-security-group --group-name sg-rds-sql --description "RDS SQL Server" --vpc-id vpc-0123abcd
aws ec2 authorize-security-group-ingress --group-id sg-0db00001 --protocol tcp --port 1433 \
  --source-group sg-0app0002
# Optional: allow DBA workstations or on-premises range through VPN/Direct Connect
aws ec2 authorize-security-group-ingress --group-id sg-0db00001 --protocol tcp --port 1433 \
  --cidr 10.20.0.0/16
```

Do not make it public

Keep `PubliclyAccessible` set to **false**. Reach the database from your application subnets and over VPN or Direct Connect. If a developer needs ad-hoc access, use a bastion host or AWS Systems Manager Session Manager port forwarding rather than opening 1433 to the internet.

4.3 Step 2: Parameter group, option group and IAM role

Parameter group (replaces sp_configure for many settings)

The default parameter group cannot be edited, so create your own, adjust settings, and attach it to the instance. Dynamic parameters apply immediately; static ones need a reboot.

Parameter (examples)	Purpose	Typical starting point
max degree of parallelism	Limits parallel threads per query	4 or 8 for OLTP; match NUMA/cores
cost threshold for parallelism	When a query becomes eligible for parallel plan	25-50 instead of the default 5
optimize for ad hoc workloads	Reduces plan-cache bloat from single-use queries	1 (on) for apps with many ad hoc queries
max server memory (mb)	SQL Server memory cap	Leave RDS default unless advised
backup compression default	Compress native backups by default	1 (on)
rds.force_ssl	Force TLS on all connections	1 for production (test clients first)
contained database authentication	Allow contained database users	As required by design
clr enabled	Allow CLR assemblies	Only if you use CLR (check restrictions)

```
aws rds create-db-parameter-group --db-parameter-group-name sql2022-se-params \
  --db-parameter-group-family sqlserver-se-16.0 --description "Production SQL Server 2022 SE"

aws rds modify-db-parameter-group --db-parameter-group-name sql2022-se-params --parameters \
  "ParameterName='max degree of parallelism',ParameterValue=4,ApplyMethod=immediate" \
  "ParameterName='cost threshold for parallelism',ParameterValue=50,ApplyMethod=immediate"
```

Option group (adds features)

Option	What it enables	Notes
SQLSERVER_BACKUP_RESTORE	Native .bak backup and restore to/from S3	Needs an IAM role with S3 access. Essential for migrations.
TDE	Transparent Data Encryption inside SQL Server	Separate from KMS storage encryption; supports certificate backup.
SQLSERVER_AUDIT	SQL Server Audit written to S3 / log targets	Compliance auditing.
S3_INTEGRATION	Copy files between S3 and the instance's D: drive	Bulk loads and file transfers without local disk access.
SSIS / SSRS / SSAS	Integration, Reporting and Analysis Services	Run on the RDS host with restrictions; confirm version support.
MSDTC	Distributed transactions	Needs additional network and parameter setup.

```
aws rds create-option-group --option-group-name sql2022-se-options --engine-name sqlserver-se \
  --major-engine-version 16.00 --option-group-description "Native backup + TDE"

aws rds add-option-to-option-group --option-group-name sql2022-se-options --apply-immediately \
  --options "OptionName=SQLSERVER_BACKUP_RESTORE,OptionSettings=[{Name=IAM_ROLE_ARN,Value=arn:aws:iam::111122223333:role/
```

IAM role for native backup

Create an IAM role that RDS can assume, trusted by `rds.amazonaws.com`, with permission to list the bucket and get/put objects (and KMS permissions if you encrypt backups). Use a bucket in the **same Region** as the instance.

4.4 Step 3: Create the DB instance

In the console choose **Create database > Standard create > Microsoft SQL Server**. The most important fields are below; the CLI example follows.

Console section	Setting	Recommendation
Engine options	Edition and version	Standard/Enterprise; latest supported minor version
Templates	Production / Dev-Test	Production enables Multi-AZ by default
Settings	Master username and password	Let RDS manage the password in Secrets Manager
Instance configuration	DB instance class	From the sizing exercise (chapter 3)
Storage	Type, size, provisioned IOPS, autoscaling	gp3 or io2; enable autoscaling with a safe maximum
Availability	Multi-AZ deployment	Yes for production
Connectivity	VPC, DB subnet group, public access, security group	Private subnets, public access = No
Authentication	Password and/or Microsoft AD	Add AWS Managed Microsoft AD for Windows auth
Monitoring	Performance Insights, Enhanced Monitoring, log exports	Enable all; export error and agent logs to CloudWatch
Additional configuration	Collation, time zone, parameter/option group, backup, encryption, maintenance, deletion protection	Match source collation; set these at creation

```
aws rds create-db-instance --db-instance-identifier sql-prod-01 \
  --engine sqlserver-se --engine-version <supported-version> --license-model license-included \
  --db-instance-class db.m6i.2xlarge --storage-type gp3 --allocated-storage 500 \
  --max-allocated-storage 1500 --storage-encrypted --kms-key-id alias/rds-sql \
  --master-username sqladmin --manage-master-user-password \
  --multi-az --no-publicly-accessible --db-subnet-group-name sql-subnets \
  --vpc-security-group-ids sg-0db00001 \
  --db-parameter-group-name sql2022-se-params --option-group-name sql2022-se-options \
  --character-set-name SQL_Latin1_General_CP1_CI_AS --timezone "India Standard Time" \
  --backup-retention-period 14 --preferred-backup-window 21:00-22:00 \
  --preferred-maintenance-window sun:22:30-sun:23:30 --auto-minor-version-upgrade \
  --deletion-protection --enable-performance-insights \
  --enable-cloudwatch-logs-exports error agent
```

Set these at creation time

Collation and **time zone** are fixed when the instance is created (changing them later is restricted or not possible, so verify in the current docs). **Storage encryption** also cannot be switched on afterwards. Getting them wrong means rebuilding the instance and migrating again.

4.5 Step 4: Connect and verify

- Find the endpoint on the instance's **Connectivity** tab (`name.xxxxxxxx.region.rds.amazonaws.com`).
- In SSMS use **Server name = endpoint,1433** (comma, not colon). Use SQL Authentication with the master user for the first connection.
- Make sure your client's network path is open: security group, route tables, VPN/Direct Connect, and any on-premises firewall.

```
-- Verify what you got
SELECT @@VERSION;
SELECT SERVERPROPERTY('Edition') AS Edition, SERVERPROPERTY('ProductVersion') AS Version,
       SERVERPROPERTY('Collation') AS Collation;

-- Is this instance Single-AZ or Multi-AZ, and which technology?
EXEC msdb.dbo.rds_get_high_availability_option;

-- Check the connection is encrypted
SELECT session_id, encrypt_option FROM sys.dm_exec_connections WHERE session_id = @@SPID;
```

4.6 Windows Authentication (optional)

RDS can join the instance to an **AWS Managed Microsoft AD** (or work with a self-managed AD through trust) so users sign in with their domain accounts using Kerberos. Setup outline: create the directory in the same VPC, create an IAM role for RDS to join the domain, select the directory on the instance, then create Windows logins in SQL Server with `CREATE LOGIN [DOMAIN\\user] FROM WINDOWS`. Name resolution and Kerberos SPNs must be correct for the endpoint.

4.7 Step 5: Day-one hardening checklist

Done?	Task
[ ]	Create application logins with least privilege (never use the master user in applications)
[ ]	Create databases, set compatibility level, confirm Query Store is on
[ ]	Create SQL Agent maintenance jobs (index/statistics, DBCC CHECKDB, cleanup) - for example the Ola Hallengren scripts
[ ]	Set CloudWatch alarms: CPU, free storage, latency, connections, replica lag (chapter 8)
[ ]	Subscribe to RDS event notifications through SNS (failover, maintenance, low storage)
[ ]	Take a manual snapshot as a baseline and test a point-in-time restore into a new instance
[ ]	Perform a controlled Multi-AZ failover test (chapter 6) and measure application recovery
[ ]	Document endpoint, parameter/option groups, security groups, contacts and the runbook
[ ]	Enable deletion protection and tag the instance (Environment, Owner, CostCentre)

5. Security

Secure RDS the same way you secure SQL Server on-premises, but with AWS tools: private networking, tight identity control, encryption everywhere, and audit trails.

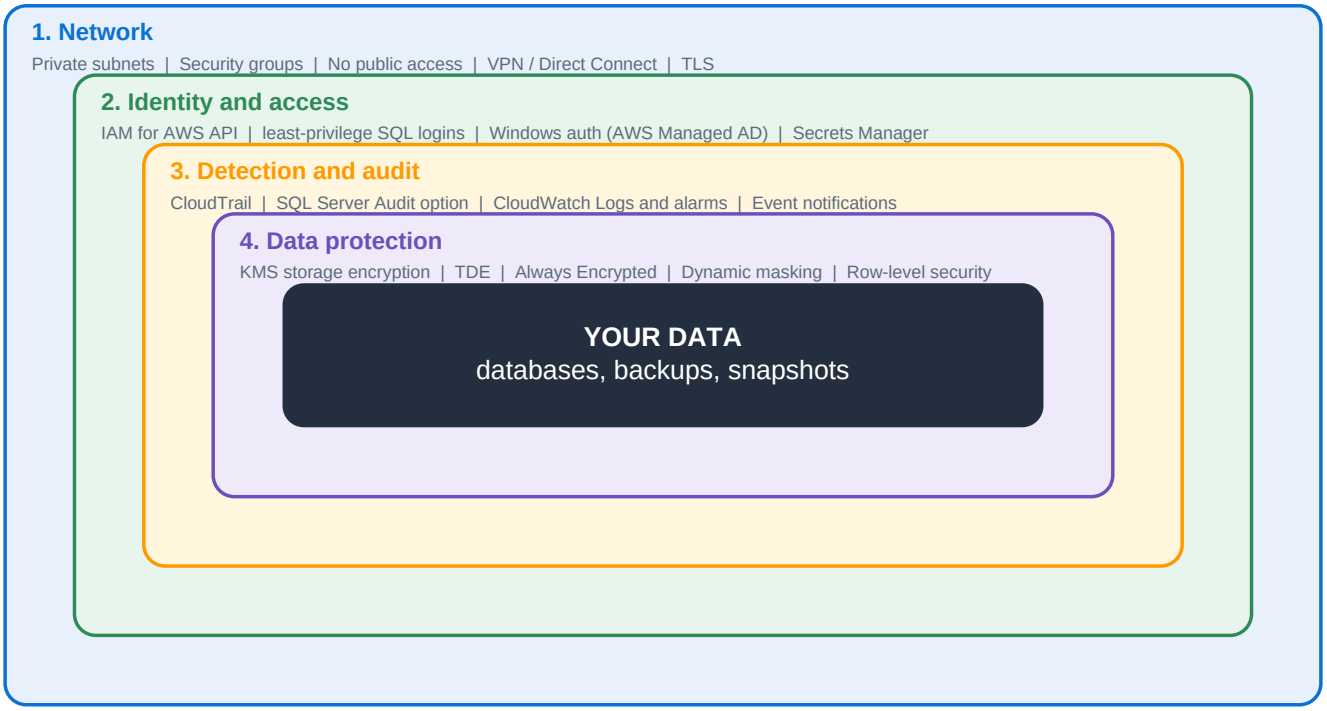


Figure 5.1 - Defence in depth for RDS for SQL Server

5.1 Network security

- Deploy in **private subnets** with **public access disabled**.
- Use **security groups** as the main firewall: reference the application's security group instead of IP ranges wherever possible.
- Reach on-premises through **Site-to-Site VPN** or **Direct Connect**; use VPC peering or Transit Gateway between VPCs.
- For admin access from outside, use a **bastion host** or **Session Manager** port forwarding, not open ports.
- Add an **RDS Proxy** (where supported for SQL Server) to pool connections and speed up failover recovery for busy applications.

5.2 Authentication and authorization

Layer	How it works on RDS	Advice
AWS control plane (IAM)	IAM policies decide who may create, modify, reboot, snapshot or delete instances	Separate roles for DBAs, developers, auditors; require MFA; use tags and conditions
SQL Server logins	SQL authentication (master user plus your logins) or Windows logins through AWS Managed AD	Prefer Windows/AD groups; strong passwords; rotate; avoid sharing the master user
Database permissions	Standard SQL Server roles and GRANT/DENY	Least privilege; db_datareader/db_datawriter and schema-level GRANTs for apps
Master user password	Can be managed in AWS Secrets Manager with automatic rotation	Turn on 'Manage master credentials in Secrets Manager'

IAM database authentication (token-based login) is not available for RDS for SQL Server. Use SQL logins or Windows Authentication.

```
-- Application login with only the permissions it needs
CREATE LOGIN app_sales WITH PASSWORD = '<long-random-password>', CHECK_POLICY = ON;
USE SalesDB;
CREATE USER app_sales FOR LOGIN app_sales;
ALTER ROLE db_datareader ADD MEMBER app_sales;
ALTER ROLE db_datawriter ADD MEMBER app_sales;
GRANT EXECUTE ON SCHEMA::dbo TO app_sales;
```

5.3 Encryption

Where	Feature	Details
At rest (storage)	KMS-based EBS encryption	Choose a customer-managed KMS key. Encrypts data files, logs, backups, snapshots and replicas. Must be chosen at creation.
At rest (database)	TDE option	SQL Server TDE via option group, with certificate backup/restore procedures. Useful when you must carry the encrypted database elsewhere or satisfy specific compliance wording.
In transit	TLS (SSL)	RDS provides a server certificate. Set `rds.force_ssl=1`, install the RDS CA bundle on clients, and rotate when certificates are renewed.
In use	Always Encrypted	Client-side encryption of sensitive columns; keys held outside the database (for example in AWS KMS-backed stores or other key stores).
Native backups	KMS key on .bak files in S3	Provide a KMS key ARN to backup and restore procedures; apply bucket encryption and policies.

5.4 Protecting data inside the database

- **Dynamic Data Masking** and **Row-Level Security** work as in any SQL Server (same T-SQL as on-premises).
- **Column-level permissions** and **schemas** separate sensitive tables from general ones.
- **Data classification** through SSMS can label columns for compliance reports.

5.5 Auditing and detection

- **CloudTrail** - records every RDS API call (who created, modified, deleted, restored). Send to a protected S3 bucket.
- **SQL Server Audit** - enable the SQLSERVER_AUDIT option to write server audit files to S3 (or to CloudWatch logs where supported). Audit logins, permission changes, DDL and access to sensitive tables.
- **CloudWatch Logs** - export the SQL Server error log and agent log; create metric filters and alarms for failed logins or severity 17+ errors.
- **RDS event subscriptions** - get SNS alerts on failover, configuration changes and low storage.
- **AWS Config and Security Hub** - continuous checks such as 'RDS storage is encrypted' or 'instance is not public'.

5.6 Security checklist

Done?	Control	Why
[ ]	Private subnets, PubliclyAccessible = false	No internet exposure
[ ]	Security group allows 1433 only from app SG and known ranges	Smallest possible attack surface
[ ]	KMS encryption at rest with a customer-managed key	Key control and audit
[ ]	rds.force_ssl = 1 and clients validate certificates	Protect data in transit
[ ]	Master password in Secrets Manager; master user not used by apps	Credential hygiene
[ ]	Windows/AD authentication or strong SQL logins with least privilege	Limit damage from stolen credentials
[ ]	CloudTrail on; SQL Audit enabled for sensitive systems	Investigation and compliance
[ ]	Error and agent logs exported to CloudWatch with alarms	Early warning
[ ]	Deletion protection, tagging, IAM guard-rails (SCPs/policies)	Prevent accidents
[ ]	Snapshots encrypted; sharing and copying restricted	Avoid data leakage via snapshots

6. High Availability, Multi-AZ and Disaster Recovery

High availability (HA) keeps the database running through everyday failures such as a failed host or an AZ problem. **Disaster recovery (DR)** restores service after a bigger event such as the loss of a whole Region. RDS gives you both, but they are different features and you must choose them deliberately.

6.1 Key terms

Term	Meaning	Example
RPO	How much data you can afford to lose (time)	'No more than 5 minutes of data'
RTO	How long you can be down	'Back within 30 minutes'
High availability	Automatic recovery inside a Region	Multi-AZ failover to the standby
Disaster recovery	Recovery after a Region-level event	Cross-Region read replica or copied backups
Backup	Protection against deletion and corruption	Point-in-time restore, snapshots

6.2 How Multi-AZ works

In a **Multi-AZ deployment**, RDS creates a standby DB instance in a different Availability Zone and keeps it in sync with the primary using **synchronous replication**. Every committed transaction is hardened on both copies. If the primary fails, RDS promotes the standby and updates the DNS name, so your applications reconnect to the same endpoint.

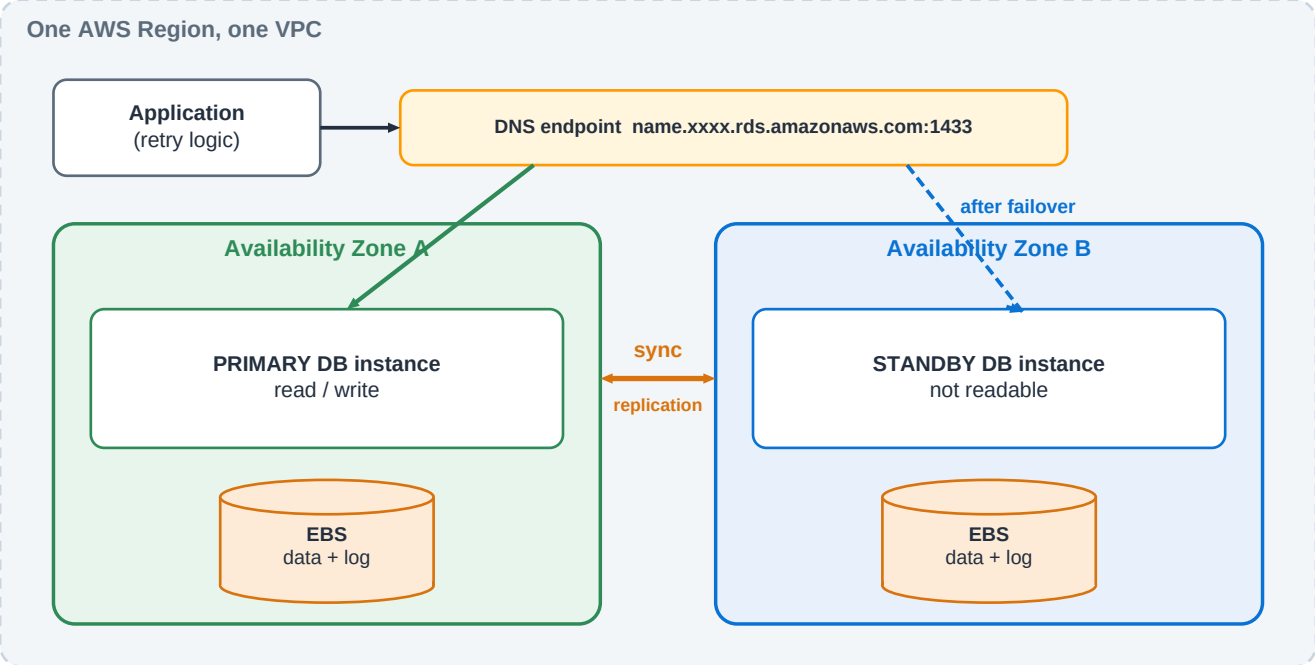


Figure 6.1 - Multi-AZ: primary and synchronous standby in two Availability Zones

- **Automatic** - RDS monitors health and handles failover without scripts. You manage nothing on the standby.
- **All databases** - every user database on the instance is protected automatically.
- **Only when in sync** - failover happens only if the standby is fully synchronised, so there is no data loss.

- **The standby is not readable** - it is for availability, not for reporting. Use read replicas for read scale-out (Enterprise).
- **Lower-impact maintenance** - operating-system and many instance changes are applied to the standby first, followed by a failover, which shortens downtime.

Which technology does RDS use?

You do not choose; RDS selects the technology based on edition and version. Knowing which one you have helps with troubleshooting and feature support.

Technology	Used for (as documented at time of writing)	Notes
Always On Availability Groups	SQL Server 2025 and 2022 (Standard, Enterprise); 2019 Standard (newer minor versions) and Enterprise; 2017 Standard/Enterprise (newer minor versions); 2016 Enterprise (newer minor versions)	Preferred technology; also the basis for read replicas on Enterprise
Database Mirroring (DBM)	Older minor versions of 2016/2017/2019 Standard and some earlier builds	Deprecated by Microsoft; upgrade minor version to use AGs. In-memory OLTP may need to be disabled first on 2016/2017 Enterprise before enabling Multi-AZ
Block-level replication	SQL Server 2022 Web Edition (recent minor versions)	Storage-level synchronisation between primary and standby

Version cut-offs change with each minor release. Confirm in the 'Multi-AZ deployments for Amazon RDS for SQL Server' page. Express edition does not support Multi-AZ.

```
-- Which Multi-AZ technology is this instance using?
EXEC msdb.dbo.rds_get_high_availability_option;
-- Returns: Multi-AZ (AlwaysOn) | Multi-AZ (Mirroring) | Multi-AZ (Block level replication) | Single-AZ
```

6.3 What happens during a failover

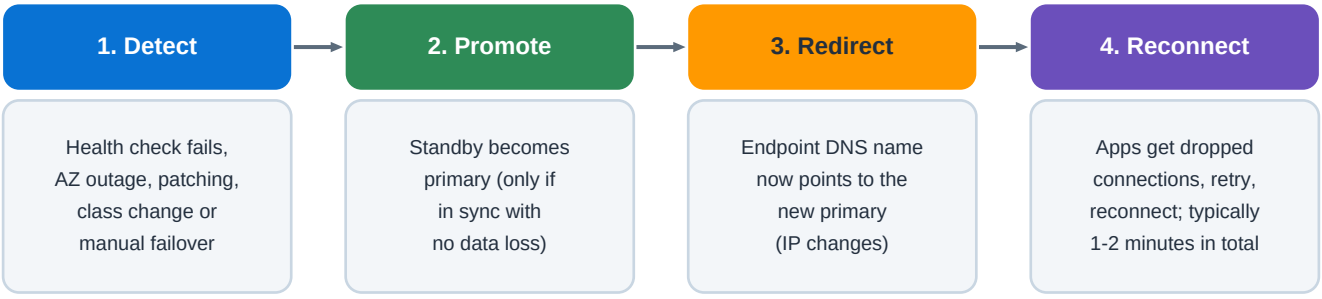


Figure 6.2 - The failover sequence

Failover is triggered by: a failed instance or host, an AZ problem, storage failure, a change that requires a restart (for example a class change or OS patch) or when *you* force it for testing. Expect existing connections to drop and a short window (commonly one to two minutes, sometimes longer when the new primary has to recover large transactions) before the endpoint accepts connections again.

After a failover, check these

- Applications reconnected (look for retry storms or pools holding dead connections).
- **Logins and SQL Agent jobs** - AG-based Multi-AZ keeps logins in sync; agent job replication may need to be enabled using the RDS procedure for system object sync. Test it; do not assume.
- Replica lag on read replicas returned to normal.
- RDS events (`aws rds describe-events`) for the reason and timeline.

6.4 Configuring Multi-AZ

You can choose Multi-AZ at creation or convert an existing instance. Converting is online, but RDS takes a snapshot and builds the standby, so I/O latency can rise and the operation can take a while on large databases. Do it in a quiet period.

```
# Convert a Single-AZ instance to Multi-AZ (use --apply-immediately or wait for the maintenance window)
aws rds modify-db-instance --db-instance-identifier sql-prod-01 --multi-az --apply-immediately

# Go back to Single-AZ
aws rds modify-db-instance --db-instance-identifier sql-prod-01 --no-multi-az --apply-immediately

# TEST: force a failover (this WILL interrupt connections; do it in a drill window)
aws rds reboot-db-instance --db-instance-identifier sql-prod-01 --force-failover
```

Multi-AZ rules to remember

Databases cannot be renamed while the instance is Multi-AZ.
A Multi-AZ instance cannot be stopped.
The standby does not serve reads and is not a backup: deleted data is deleted on both sides.
Place the instance in a DB subnet group that has subnets in at least two AZs.

6.5 Making applications failover-ready

- Always connect using the **endpoint DNS name**, never an IP address. Keep client DNS caching short.
- Use **retry logic with back-off** for transient errors, and test that connection pools discard broken connections.
- Set sensible **connection and command timeouts** (for example connect timeout 15-30 seconds) and keep transactions short and idempotent.
- Consider **RDS Proxy** (where supported for SQL Server) to keep client connections stable and speed up recovery.
- Rehearse: run **--force-failover** in a pre-production environment under load and record the real application downtime.

6.6 Read replicas (Enterprise Edition)

A **read replica** is an asynchronous, readable copy of the primary. RDS builds it with Always On technology (a **distributed availability group** for cross-Region copies). Use replicas to offload reporting, to build a DR copy in another Region, or to prepare for migrations between Regions or accounts.

Topic	Details (verify current limits)
Edition and version	Enterprise Edition only; Always On-based versions (documented for SQL Server 2016 through 2022; check support for newer versions)
Source	Multi-AZ source, and also Single-AZ Enterprise sources since 2024; automated backups must be enabled (retention not 0)

Topic	Details (verify current limits)
Number	Several replicas per source; the maximum differs between in-Region and cross-Region and has increased over time
Cross-Region	Supported; you must supply a KMS key from the target Region for encrypted sources
Not supported on replicas	Backups/point-in-time restore from the replica, manual snapshots, Multi-AZ replicas, replicas of replicas
Failover	Promote manually to a standalone instance; you cannot 'fail back' by re-attaching the old primary
Logins and jobs	Cross-Region replicas do not receive new logins or jobs created after replica creation; script and re-apply them
Native restore on source	Native restores on the source are blocked while cross-Region replicas exist

```
# In-Region read replica
aws rds create-db-instance-read-replica --db-instance-identifier sql-prod-01-rr1 \
  --source-db-instance-identifier sql-prod-01 --db-instance-class db.m6i.xlarge

# Cross-Region read replica (run in the TARGET Region, provide a KMS key from that Region)
aws rds create-db-instance-read-replica --region ap-southeast-1 \
  --db-instance-identifier sql-prod-01-dr \
  --source-db-instance-identifier arn:aws:rds:ap-south-1:111122223333:db:sql-prod-01 \
  --kms-key-id alias/rds-sql-sg --db-instance-class db.m6i.2xlarge

# Disaster: promote the replica to a standalone instance
aws rds promote-read-replica --region ap-southeast-1 --db-instance-identifier sql-prod-01-dr
```

6.7 Disaster recovery options

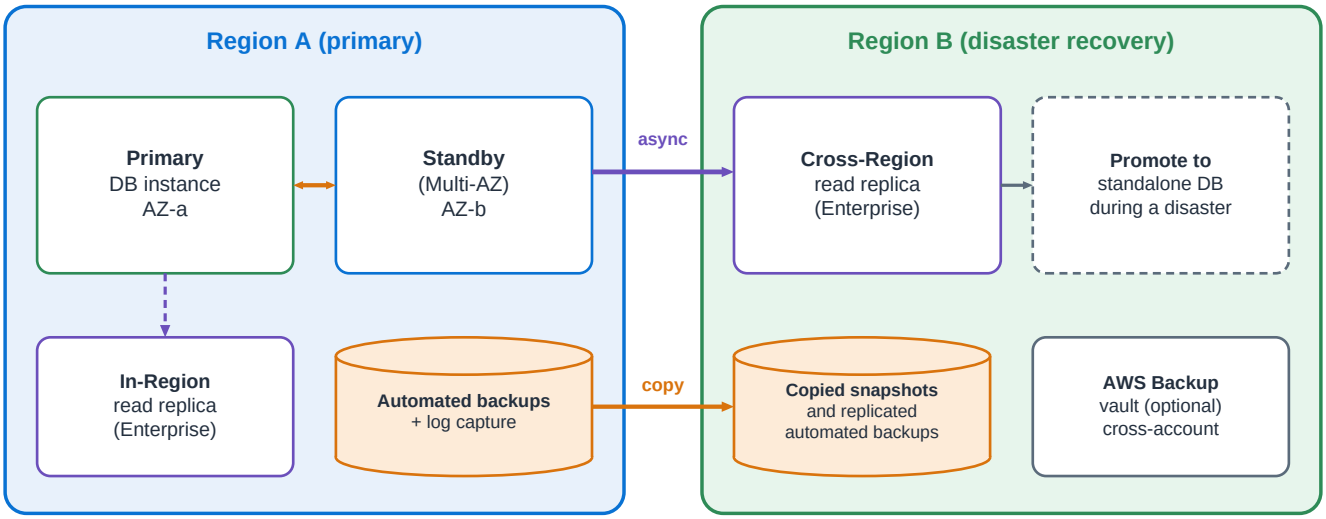


Figure 6.3 - Disaster recovery building blocks across two Regions

Option	Protects against	RPO (typical)	RTO (typical)	Cost
Multi-AZ	Instance or AZ failure	Zero (synchronous)	Minutes (automatic)	About 2x instance
Cross-Region read replica	Region failure	Seconds to minutes (async)	Minutes to tens of minutes (manual promote)	Second instance + transfer
Cross-Region automated backups	Region failure, low cost	About 5 minutes of logs plus delay	Hours (restore to new instance)	Backup storage + transfer

Option	Protects against	RPO (typical)	RTO (typical)	Cost
Copied snapshots / AWS Backup copies	Region or account loss, ransomware	Snapshot interval	Hours	Lowest
Native backup copies in S3 (CRR)	Portable DR, cross-account	Backup interval	Hours	Low

RPO and RTO depend on workload, database size and network. Measure them in drills; the table gives planning orders of magnitude only.

6.8 Choosing a design

Requirement	Design
Dev/test, can tolerate an outage	Single-AZ + automated backups
Production, survive AZ and host failure	Multi-AZ
Production plus reporting offload	Multi-AZ + in-Region read replica (Enterprise)
Survive a Region outage with low RTO	Multi-AZ + cross-Region read replica (Enterprise) or a Region-level rebuild runbook
Survive Region loss on a budget	Cross-Region automated backups or snapshot copies + tested restore runbook
Protect from ransomware / account compromise	AWS Backup with cross-account vault and immutable (vault lock) copies

Remember

Multi-AZ replicates mistakes instantly. A dropped table disappears from both copies. You need **both** HA (Multi-AZ) and backups with point-in-time restore.

7. Backup, Restore and Maintenance

RDS backs up your instance automatically, so the DBA's job shifts from 'run the backup' to 'choose retention, prove restores work and control maintenance'.

7.1 Automated backups

When automated backups are enabled (retention period 1 to 35 days), RDS takes a daily **storage snapshot** in your backup window and captures **transaction logs about every five minutes**. Together they let you restore to any second within the retention window, up to the latest restorable time (a few minutes ago).

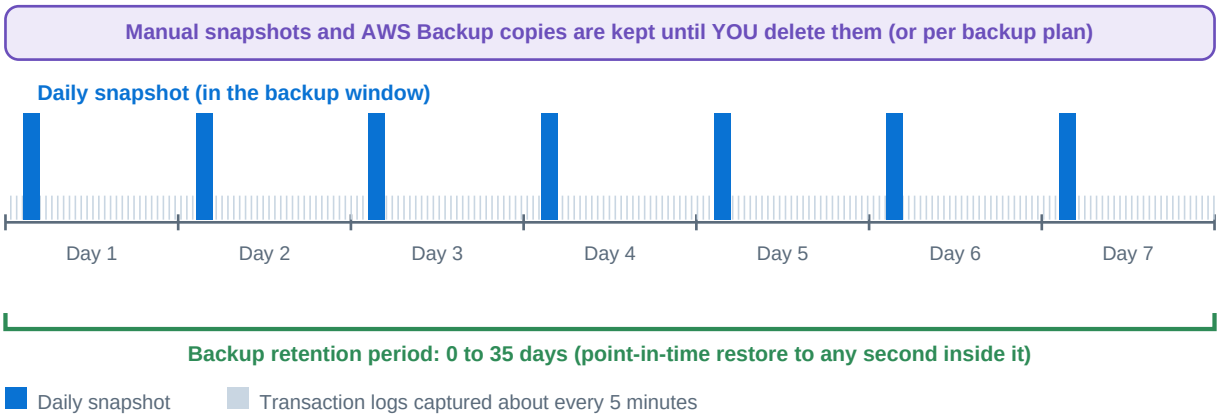


Figure 7.1 - Automated backups, log capture and retention

- Retention **0** turns automated backups off. That also disables point-in-time restore and read replicas. Do not do this in production.
- Backups are stored in AWS-managed S3 (you do not see the bucket). You can replicate them to another Region.
- **Recovery model:** point-in-time restore relies on databases being in the **FULL** recovery model. RDS enforces this when automated backups are on; a database switched to SIMPLE is not covered by PITR.
- In Single-AZ instances, a snapshot can cause a brief I/O pause; schedule the backup window outside peak hours.
- You cannot run `BACKUP LOG` or `BACKUP DATABASE ... TO DISK` yourself. For portable backups use native backup to S3 (section 7.4).

7.2 Manual snapshots and copies

- **Manual snapshots** are kept until you delete them. Take one before major changes (upgrades, migrations, schema releases).
- **Copy snapshots** to another Region for DR, or **share** them with another account. Encrypted snapshots need KMS key sharing.
- **AWS Backup** centralises schedules, retention, cross-Region and cross-account copies and vault lock for RDS.
- A **final snapshot** on deletion is strongly recommended; also keep **deletion protection** on.

7.3 Restoring

Every RDS restore creates a **new DB instance** with a **new endpoint**. It never overwrites the original. After restoring you either repoint applications, rename instances to swap, or copy specific data back.

Scenario	Method	Notes
Bad deployment or accidental delete a few hours ago	Point-in-time restore to a new instance	Pick a time just before the mistake; extract the data and copy it back (or swap instances)
Whole instance lost or corrupted	Restore from the latest snapshot / point in time	Assign the correct parameter group, option group, security group, subnet group
Need one database back from the instance	Restore the instance to a new one and copy that database; or restore a native .bak if you keep them	RDS has no single-database snapshot restore
Region outage	Restore from copied snapshots or replicated backups, or promote a cross-Region replica	Plan KMS keys and networking in the DR Region
Clone for testing	Restore a snapshot to a Dev instance	Mask or scrub production data

```
# Point-in-time restore to a NEW instance (explicitly set groups; they are not inherited automatically)
aws rds restore-db-instance-to-point-in-time --source-db-instance-identifier sql-prod-01 \
  --target-db-instance-identifier sql-prod-01-restore --restore-time 2026-10-28T10:15:00Z \
  --db-parameter-group-name sql2022-se-params --option-group-name sql2022-se-options \
  --vpc-security-group-ids sg-0db00001 --db-subnet-group-name sql-subnets

# Restore from a snapshot
aws rds restore-db-instance-from-db-snapshot --db-instance-identifier sql-prod-01-restore \
  --db-snapshot-identifier sql-prod-01-before-release --db-instance-class db.m6i.2xlarge
```

> **Test restores**
> Schedule a restore drill at least every quarter. Time it, validate the data (row counts, key reports), and record the steps. A restore that has never been tested is only an assumption.

7.4 Native backup and restore with S3

Native backup (.bak files) is how you move databases in and out of RDS and how you keep *portable* copies. It needs the **SQLSERVER_BACKUP_RESTORE** option and an IAM role (section 4.3). The procedures live in the `msdb` database. Detailed migration use is in chapters 9 and 10.

Procedure	Purpose
msdb.dbo.rds_backup_database	Back up a database to S3 (full or differential)
msdb.dbo.rds_restore_database	Restore a full or differential backup from S3 (WITH or WITHOUT recovery)
msdb.dbo.rds_restore_log	Apply a transaction log backup from S3 (enables log-shipping style migrations)
msdb.dbo.rds_finish_restore	Bring a database that was restored with NORECOVERY online
msdb.dbo.rds_task_status	Check progress of a backup or restore task
msdb.dbo.rds_cancel_task	Cancel a running task

7.5 Maintenance and upgrades

- **Maintenance window** - a weekly slot (for example Sunday 22:30-23:30) when RDS applies OS and engine updates. In Multi-AZ, patches go to the standby first, then a failover swaps roles.
- **Minor version upgrades** - security fixes and cumulative updates. Turn on auto minor version upgrade for non-production; for production, review and apply in the window you choose.
- **Major version upgrades** (for example 2019 to 2022) - take a snapshot, test on a restored copy, move to a new parameter/option group family, then upgrade. Keep the old compatibility level until testing says otherwise, then raise it with Query Store protection.
- **Scaling** - changing instance class or storage is a modification. Choose 'apply immediately' only in a quiet period; Multi-AZ shortens the outage to a failover.
- **Stop/start** - Single-AZ instances can be stopped for a limited time to save cost; storage is still billed.

Major upgrade checklist

Step	Action
1	Read the release notes and run an assessment for deprecated or discontinued features
2	Create new parameter/option groups for the target version family
3	Take a manual snapshot, restore it as a test instance and upgrade the test first
4	Run application regression and performance tests (Query Store comparison)
5	Upgrade production in a maintenance window; keep compatibility level, then update statistics
6	Monitor closely, then raise compatibility level in a separate change

8. Monitoring and Day-to-Day Operations

Monitoring RDS combines AWS-level metrics (the host, storage and instance health) with the SQL Server tools you already use for query and wait analysis.

8.1 The monitoring toolbox

Tool	What it shows	When to use
CloudWatch metrics	CPU, memory, storage, IOPS, latency, connections, replica lag (1-minute granularity)	Dashboards and alarms
Enhanced Monitoring	OS-level metrics per process, down to 1-second granularity	Find which process or thread is consuming resources
Performance Insights / Database Insights	Database load (average active sessions) by wait, SQL, user, host	Find the queries and waits behind a slowdown. AWS is evolving this into CloudWatch Database Insights; check the current console.
RDS events and SNS	Failover, maintenance, backup, low storage, configuration changes	Operational alerting
Logs in CloudWatch	SQL Server error log, agent log, audit	Search, metric filters, alarms
SQL Server tools	DMVs, Query Store, Extended Events, wait stats	Deep query-level troubleshooting
CloudTrail	API activity	Who changed what in AWS

8.2 Metrics every DBA should watch

Metric (CloudWatch)	Meaning	Typical alarm
CPUUtilization	Host CPU use	> 80% for 15 minutes
FreeableMemory	Memory available on the host	Sustained very low value (SQL uses most memory by design; trend matters)
FreeStorageSpace	Free space on the data volume	< 15-20% free or < N GB
ReadLatency / WriteLatency	Disk latency per I/O	> 10-20 ms sustained (gp3); lower for io2
ReadIOPS / WriteIOPS	I/O operations per second	Near provisioned or instance EBS limit
DiskQueueDepth	Pending I/O requests	Sustained high values mean storage saturation
DatabaseConnections	Active connections	Near expected maximum or sudden drops
Network throughput	Bytes in/out	Near instance network limit
EBSIOBalance% / EBSByteBalance%	Burst balance of the instance's EBS bandwidth	Falling toward zero on burstable or smaller classes
BurstBalance (gp2)	Remaining I/O credits for gp2 volumes	Low value: move to gp3
ReplicaLag	How far a read replica is behind	> a few seconds sustained (business dependent)

```
aws cloudwatch put-metric-alarm --alarm-name sql-prod-01-cpu-high --namespace AWS/RDS \
  --metric-name CPUUtilization --dimensions Name=DBInstanceIdentifier,Value=sql-prod-01 \
  --statistic Average --period 300 --evaluation-periods 3 --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:ap-south-1:111122223333:dba-alerts
```

8.3 Logs and the SQL Server error log

Export the **error** and **agent** logs to CloudWatch Logs so they are searchable and survive instance replacement. You can also read the error log from T-SQL.

```
-- Read the current error log (RDS helper procedure)
EXEC rdsadmin.dbo.rds_read_error_log @index = 0, @type = 1;
```

8.4 Finding what is slow inside SQL Server

```
-- What is running right now, and what is it waiting for?
SELECT r.session_id, r.status, r.wait_type, r.wait_time, r.blocking_session_id,
       r.cpu_time, r.logical_reads, SUBSTRING(t.text, 1, 200) AS sql_text
FROM sys.dm_exec_requests r CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id > 50 AND r.session_id <> @@SPID ORDER BY r.cpu_time DESC;

-- Top waits since the last restart
SELECT TOP 10 wait_type, wait_time_ms / 1000.0 AS wait_sec, waiting_tasks_count
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN ('SLEEP_TASK','LAZYWRITER_SLEEP','XE_TIMER_EVENT','REQUEST_FOR_DEADLOCK_SEARCH',
      'CHECKPOINT_QUEUE','DIRTY_PAGE_POLL','SQLTRACE_INCREMENTAL_FLUSH_SLEEP','BROKER_TO_FLUSH')
ORDER BY wait_time_ms DESC;

-- Why can't the log be reused? (log growth or storage-full investigations)
SELECT name, recovery_model_desc, log_reuse_wait_desc FROM sys.databases;
```

Use **Query Store** to find regressions and force good plans, and **Performance Insights** to see load by wait type. Popular community tools such as **sp_WhoIsActive**, **sp_Blitz** and **Ola Hallengren's maintenance solution** work on RDS as ordinary T-SQL objects.

8.5 Routine DBA tasks that are still yours

Task	Frequency	Notes
Index and statistics maintenance	Weekly / nightly	SQL Agent jobs; avoid rebuilding everything blindly
DBCC CHECKDB	Weekly (or on a restored copy)	Run on the instance or on a restored test copy to offload
Review failed jobs, error log, deadlocks	Daily	CloudWatch Logs and Agent job history
Capacity review (storage, CPU, IOPS)	Monthly	Right-size; adjust autoscaling limits
Restore drill and failover drill	Quarterly	Record real recovery times
Patch and version review	Monthly / per release	Check pending maintenance and version support dates
Security review (logins, SG rules, audit)	Quarterly	Remove unused logins and open rules

8.6 Troubleshooting guide

Symptom	Likely cause	What to check
High CPU	Bad plans, missing indexes, parallelism settings, undersized class	Performance Insights top SQL, Query Store, MAXDOP and cost threshold
High latency, low CPU	I/O bound: volume or instance EBS limits reached	Read/WriteIOPS vs limits, DiskQueueDepth, move to gp3 with more IOPS or io2
Storage full / low free space	Data growth, log growth, tempdb, replication holding log	FreeStorageSpace, log_reuse_wait_desc, long transactions, enable autoscaling
Cannot connect	Security group, route, DNS, wrong endpoint format, max connections	SG rules, VPC routes/VPN, 'host,1433', DatabaseConnections
Intermittent disconnects	Failover, maintenance, idle connection timeouts at firewalls	RDS events, retry logic, TCP keep-alive settings
Slow after migration	Compatibility level, stale statistics, different hardware profile	Update statistics, compare Query Store, tune parameters
Replica lag	Heavy write load, undersized replica, network	ReplicaLag metric, replica class, long-running queries on replica
Instance stuck in 'storage-optimization' or 'modifying'	Large storage or class change in progress	Wait; check events; avoid more changes until it completes

9. Migrating from On-Premises SQL Server to Amazon RDS

A good migration is mostly preparation. This chapter gives you the plan, the methods, a worked native backup/restore procedure with near-zero downtime, and the checks that stop surprises on cutover night.

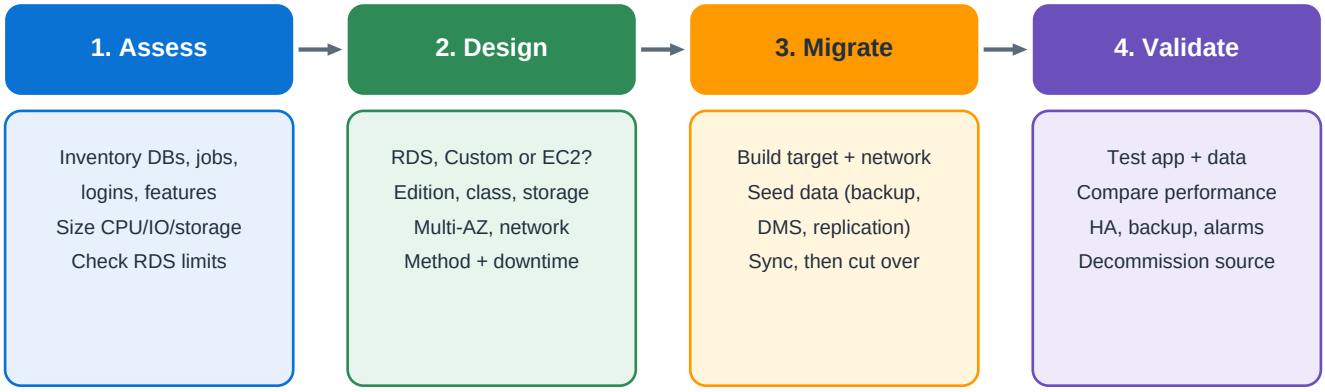


Figure 9.1 - Four phases of a database migration

9.1 Phase 1: Assess

List everything that surrounds the databases, not only the databases. RDS removes OS and server-level access, so every dependency on those must be found early.

Check	Why it matters on RDS	How to find it
Database sizes, growth, count	Storage sizing, number of DBs per instance, transfer time	sys.master_files, backup history
SQL Server version and edition features	Target version >= source; Enterprise-only features block a restore to Standard	SERVERPROPERTY, sys.dm_db_persisted_sku_features
SQL Agent jobs and steps	Jobs must be re-created; CmdExec/PowerShell steps and OS file access usually do not work	msdb.dbo.sysjobs, sysjobsteps
Logins, roles, credentials, proxies	Logins are not in restored databases; SIDs must match	sys.server_principals, sys.sql_logins
Linked servers, DB Mail, replication	Re-create and test connectivity from the VPC	sys.servers, Database Mail config
CLR, FILESTREAM, PolyBase, ML Services, Stretch DB	Check support list; some are unavailable	sys.assemblies, sys.master_files (FILESTREAM)
SSIS / SSRS / SSAS usage	Option-based support or move to EC2/other services	Inventory of packages/reports/cubes
Collation, time zone, compatibility level	Collation and time zone are fixed at instance creation	SERVERPROPERTY('Collation'), sys.databases
Performance baseline	Right-sizing and post-migration comparison	Perfmon, Query Store, wait stats
Dependencies (apps, ETL, reports)	Connection strings, firewall, DNS, authentication changes	Application inventory

```
-- Quick feature and object inventory on the source server
SELECT name, provider, data_source FROM sys.servers WHERE is_linked = 1;      -- linked servers
SELECT COUNT(*) AS agent_jobs FROM msdb.dbo.sysjobs;
SELECT DB_NAME(database_id) AS db, SUM(size) * 8 / 1024 / 1024.0 AS size_gb
FROM sys.master_files GROUP BY database_id ORDER BY size_gb DESC;
SELECT DB_NAME(database_id) AS db, name FROM sys.master_files WHERE type_desc = 'FILESTREAM';
SELECT name, permission_set_desc FROM sys.assemblies WHERE is_user_defined = 1;  -- run per database
SELECT name, type_desc, is_disabled FROM sys.server_principals WHERE type IN ('S','U','G');
```

9.2 Phase 2: Design the target

- Pick **RDS** unless a blocker exists (OS access, unsupported feature). Reconsider RDS Custom or EC2 for blockers.
- Size using the method in chapter 3. Match **collation and time zone** to the source.
- Plan **Multi-AZ**, backup retention, parameter group, option group (native backup), IAM role, KMS key, security groups.
- Plan **connectivity**: Site-to-Site VPN or Direct Connect between on-premises and the VPC.
- Decide **method and downtime** using the table below.

Method	Type	Downtime	Best for	Watch out
Native full backup + restore	Offline	Backup + copy + restore time	Small to mid DBs, simple, many DBs at once	Downtime grows with size
Native full + differential + log restore (rds_restore_log)	Online, physical	Minutes (final log + cutover)	Large DBs; same-engine, most faithful copy	You script the log copy; RDS has no built-in log shipping
AWS DMS (full load + CDC)	Online, logical	Minutes	Selective tables, continuous sync, mixed sources	Pre-create schema; LOBs; CDC prerequisites; no logins/jobs
Transactional replication	Online, logical	Minutes	Selected objects, staged cutover	Needs distributor setup; schema changes need care
BACPAC / SqlPackage	Offline	Long	Small DBs, schema + data portability	Slow for large DBs
bcp, SSIS, ADF, Glue	Offline / batch	Varies	Table-level moves and transformations	No schema/logins; you manage consistency

9.3 Moving the data: bandwidth and transfer options

Estimate transfer time early. The numbers below are ideal (100% link use); plan for 60-80% efficiency in practice.

Data size	100 Mbps	1 Gbps	10 Gbps
500 GB	about 11 hours	about 1.1 hours	about 7 minutes
5 TB	about 4.6 days	about 11 hours	about 1.1 hours
50 TB	about 46 days	about 4.6 days	about 11 hours

- **Direct upload** - `aws s3 cp` or `aws s3 sync` over Direct Connect/VPN with multipart upload.
- **AWS DataSync** - managed, scheduled, verified file transfer from on-premises to S3.
- **AWS Snowball Edge** - physical devices for tens of terabytes or poor connectivity; seed a full backup, then ship logs online.
- **Striped backups** - split very large backups into several files for parallel upload and restore.

9.4 Path A in detail: native backup + log restore (near-zero downtime)

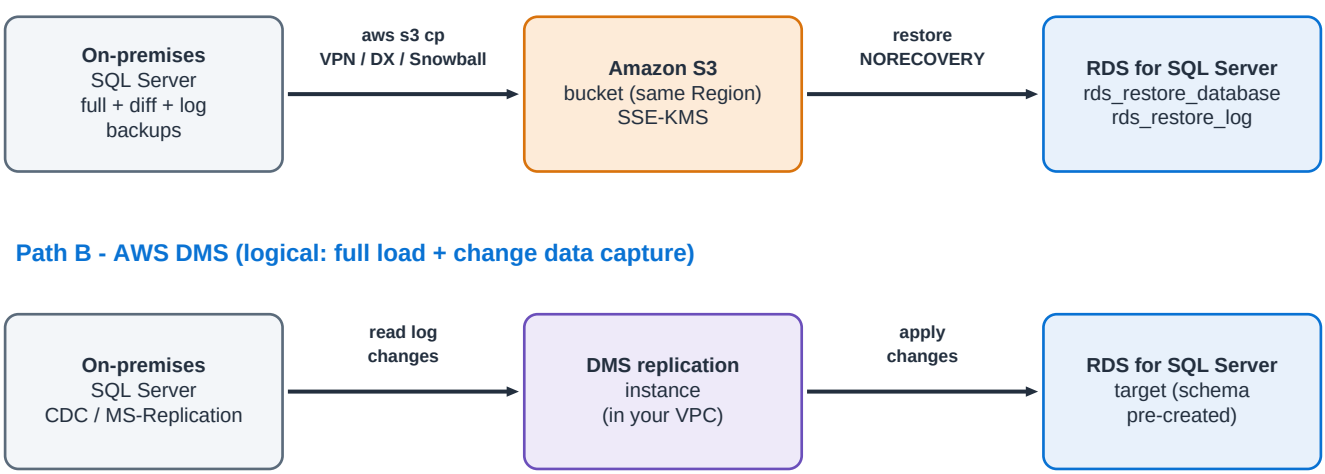


Figure 9.2 - Two migration paths into RDS for SQL Server

Step 1: Prepare AWS

- Create the S3 bucket in the **same Region** as the RDS instance; enable default encryption (SSE-KMS).
- Create the IAM role and attach the **SQLSERVER_BACKUP_RESTORE** option to the instance's option group (section 4.3).
- Create the RDS instance (start as **Single-AZ** for speed; convert to Multi-AZ after cutover if your plan allows).

Step 2: Take backups on the source and upload them

```
-- Full backup (do NOT use COPY_ONLY if you want to apply differentials later)
BACKUP DATABASE SalesDB TO DISK = 'D:\Backups\SalesDB_full.bak' WITH COMPRESSION, CHECKSUM, STATS = 10;

-- Later: differential and log backups
BACKUP DATABASE SalesDB TO DISK = 'D:\Backups\SalesDB_diff1.bak' WITH DIFFERENTIAL, COMPRESSION, CHECKSUM;
BACKUP LOG SalesDB TO DISK = 'D:\Backups\SalesDB_log_0001.trn' WITH COMPRESSION, CHECKSUM;
```

```
aws s3 cp D:\Backups\SalesDB_full.bak s3://my-migration-bucket/sales/
aws s3 sync D:\Backups\ s3://my-migration-bucket/sales/ --exclude "*" --include "*.trn" --include "*_diff*.bak"
```

Step 3: Restore the full backup on RDS WITH NORECOVERY

```
EXEC msdb.dbo.rds_restore_database
     @restore_db_name = 'SalesDB',
     @s3_arn_to_restore_from = 'arn:aws:s3:::my-migration-bucket/sales/SalesDB_full.bak',
     @with_norecovery = 1;

-- Watch progress (percent complete, status, task id)
EXEC msdb.dbo.rds_task_status @db_name = 'SalesDB';
```

Step 4: Keep the target current with differential and log backups

```
-- Latest differential (cumulative: only the most recent one is needed)
EXEC msdb.dbo.rds_restore_database @restore_db_name = 'SalesDB', @type = 'DIFFERENTIAL',
     @s3_arn_to_restore_from = 'arn:aws:s3:::my-migration-bucket/sales/SalesDB_diff1.bak',
     @with_norecovery = 1;

-- Each log backup, in sequence
EXEC msdb.dbo.rds_restore_log @restore_db_name = 'SalesDB',
     @s3_arn_to_restore_from = 'arn:aws:s3:::my-migration-bucket/sales/SalesDB_log_0001.trn',
     @with_norecovery = 1;
```

Step 5: Cut over

- Stop the application (or set the source database to `READ_ONLY`), then take the **final (tail) log backup** and upload it.
- Restore the final log with recovery to bring the database online (final `rds_restore_log` with `@with_norecovery = 0`, or `EXEC msdb.dbo.rds_finish_restore @db_name = 'SalesDB'` if the database is left in a restoring state).
- Run validation, fix orphaned users (section 9.7), repoint application connection strings, and start the application.
- Convert to Multi-AZ and enable automated backups/retention if not already enabled; take a manual snapshot.

Native restore rules to remember

The source SQL Server version must be the same as, or older than, the RDS version. You cannot restore a backup from a newer version.
A backup that uses Enterprise-only features (for example partitioning or data compression) cannot be restored to a Standard-edition RDS instance.
Differentials depend on the last full backup that is not COPY_ONLY. Do not let another tool take extra full backups during the sync period.
If the source database uses TDE, back up and restore the certificate using RDS's TDE option procedures, or remove TDE before backup.
Size and file-count limits apply (very large backups should be striped). Check current limits in the RDS user guide.
Automated RDS backups are suspended or restricted for databases in restoring state; you cannot run other restores on the same DB until finished.

9.5 Path B in detail: AWS DMS

DMS copies table data and then streams changes. It is useful when you cannot use backup/restore (for example selective tables), need long parallel running, or the source has mixed workloads. It is not a full replacement for backup/restore because it moves **data**, not logins, jobs, permissions or most programmability.

Step	Action
1	Create the target schema first (generate scripts from the source; create tables, keys, indexes, procedures on RDS). Disable triggers and foreign keys on the target during full load, then re-enable.
2	Prepare the source: enable change capture (MS-CDC on Enterprise/Standard, or MS-Replication where required); ensure the DMS user has the needed rights and the log is retained long enough.
3	Create a DMS replication instance in the VPC (size for table count and LOBs), then source and target endpoints, and test them.
4	Create a task: **full load + CDC**, with table mappings, LOB settings and validation enabled.
5	Monitor latency and errors. When latency is near zero, stop writes, let the last changes apply, validate counts/checksums, then cut over.
6	Re-create secondary objects not moved by DMS (indexes, constraints, procedures, jobs, logins) and update statistics.

Transactional replication is an alternative: initialise the subscriber (RDS) from a native backup, then replicate changes from the on-premises publisher. AWS publishes a multi-part guide for this pattern; use it when you need object-level control or a staged cutover.

9.6 Server-level objects that do not travel with the database

Object	How to migrate
Logins and passwords	Script logins with their SIDs and password hashes (for example the sp_help_revlogin pattern) and create them on RDS before or right after restore so database users match
SQL Agent jobs, operators, alerts	Script them from SSMS and re-create on RDS; replace unsupported step types (CmdExec, PowerShell) with other services such as Lambda, Systems Manager or Step Functions
Linked servers	Re-create with the right providers; test connectivity and security from the VPC
Database Mail profiles	Re-create; check SMTP connectivity and relay rules
Server-level settings (sp_configure)	Map to DB parameter group settings
Server triggers, credentials, endpoints	Re-create where supported; review what RDS allows
SSIS packages, SSRS reports, SSAS cubes	Re-host (options, EC2) or redesign; they are not part of the database restore

9.7 Phase 4: Validate and optimise

```
-- Fix orphaned users after restore (login exists, SID differs)
SELECT dp.name FROM sys.database_principals dp
LEFT JOIN sys.server_principals sp ON dp.sid = sp.sid
WHERE dp.type IN ('S','U') AND sp.sid IS NULL AND dp.authentication_type_desc = 'INSTANCE';

ALTER USER app_sales WITH LOGIN = app_sales;   -- re-map a SQL user to the login of the same name

-- After cutover: refresh statistics and set a sensible compatibility level
EXEC sp_updatestats;
ALTER DATABASE SalesDB SET COMPATIBILITY_LEVEL = 150;  -- keep the original first, then raise deliberately
```

Step	Detail
Data validation	Row counts, checksums on key tables, key reports and batch jobs compared with the source
Performance	Compare Query Store/wait stats with the baseline; tune parameter group; right-size the instance
Availability	Enable Multi-AZ, test a forced failover under load, test point-in-time restore
Security	Private access only, least-privilege logins, TLS enforced, audit enabled
Monitoring	CloudWatch alarms, Performance Insights, event subscriptions, log exports
Operations	Re-create maintenance jobs; document runbooks; hand over to support
Rollback and retire	Keep the source read-only for an agreed period, then decommission

9.8 Sample cutover runbook

When	Action	Owner
T - 2 weeks	Dress rehearsal on a production-size copy; record timings; go/no-go criteria agreed	Project lead
T - 1 day	Verify log restores are current (small lag); freeze deployments; confirm backups and rollback plan	DBA
T - 30 min	Notify stakeholders; stop schedulers and batch jobs on the source	DBA / App
T - 15 min	Put application in maintenance mode; stop writes	App team
T	Take tail-log backup, upload, restore on RDS with recovery; fix orphaned users	DBA
T + 15 min	Repoint connection strings or DNS CNAME; start application	App team
T + 30 min	Smoke tests, error log and CloudWatch review	Everyone
T + 2 hours	Go/no-go: stay on AWS, or roll back (source still intact and read-only)	Project lead
T + 1 week	Performance review, enable Multi-AZ/retention settings if deferred, decommission plan	DBA / Finance

9.9 Common pitfalls

Pitfall	Prevention
Wrong collation or time zone on the RDS instance	Check source values first; set them at creation (cannot easily change later)
Restore fails: version or edition features	Target version >= source; avoid Enterprise-only features on Standard
Missing S3 permissions or option group not attached	Test a tiny backup/restore first; confirm IAM role, bucket Region, KMS permissions
Logins missing or SIDs differ	Script logins with SIDs; fix orphaned users after restore
Agent jobs depend on OS or file paths	Redesign with AWS services; do not rely on local disk
Underestimated transfer time	Calculate bandwidth; use Snowball or DataSync when needed; start sync early
Restore into Multi-AZ slows or blocks	Restore to Single-AZ then enable Multi-AZ after cutover (if policy allows)
No rehearsal	Rehearse at least once with real data volume and timings
Performance regression after move	Baseline before; compare Query Store; tune IOPS/class; update statistics

10. Migrating from Amazon RDS to SQL Server on EC2

Sometimes you outgrow RDS or need something it does not allow. Moving from RDS to a self-managed SQL Server on EC2 gives full control and also makes you responsible for everything RDS used to do for you. This chapter covers when to do it, how to design the EC2 target and how to migrate with minimal risk.

10.1 Why move from RDS to EC2?

Reason	Comment
Need OS access, RDP or sysadmin	Third-party agents, custom drivers, OS-level tools, full server configuration
Unsupported SQL Server features or components	FILESTREAM, PolyBase, ML Services, SSRS/SSAS/SSIS co-hosted, specific CLR or file-system dependencies
Exact version or cumulative update control	Pin to a specific build or adopt a build RDS does not yet offer
Bring your own licence	Existing Software Assurance or licence-mobility benefits can lower cost (consider Dedicated Hosts and AWS License Manager)
Scale or configuration beyond RDS limits	Very large databases, number of databases, specialised storage layout
Advanced HA/DR designs	Failover cluster instances, distributed AGs to on-premises, custom log shipping

Consider RDS Custom first

If the blocker is OS or server access but you still want AWS automation for backups and patching, **RDS Custom for SQL Server** may fit and is a smaller operational step than full EC2.

10.2 Designing the EC2 target

Treat the EC2 build like a new on-premises server project, using AWS building blocks. A typical production design is a two-node Always On Availability Group spread across Availability Zones with a witness in a third location.

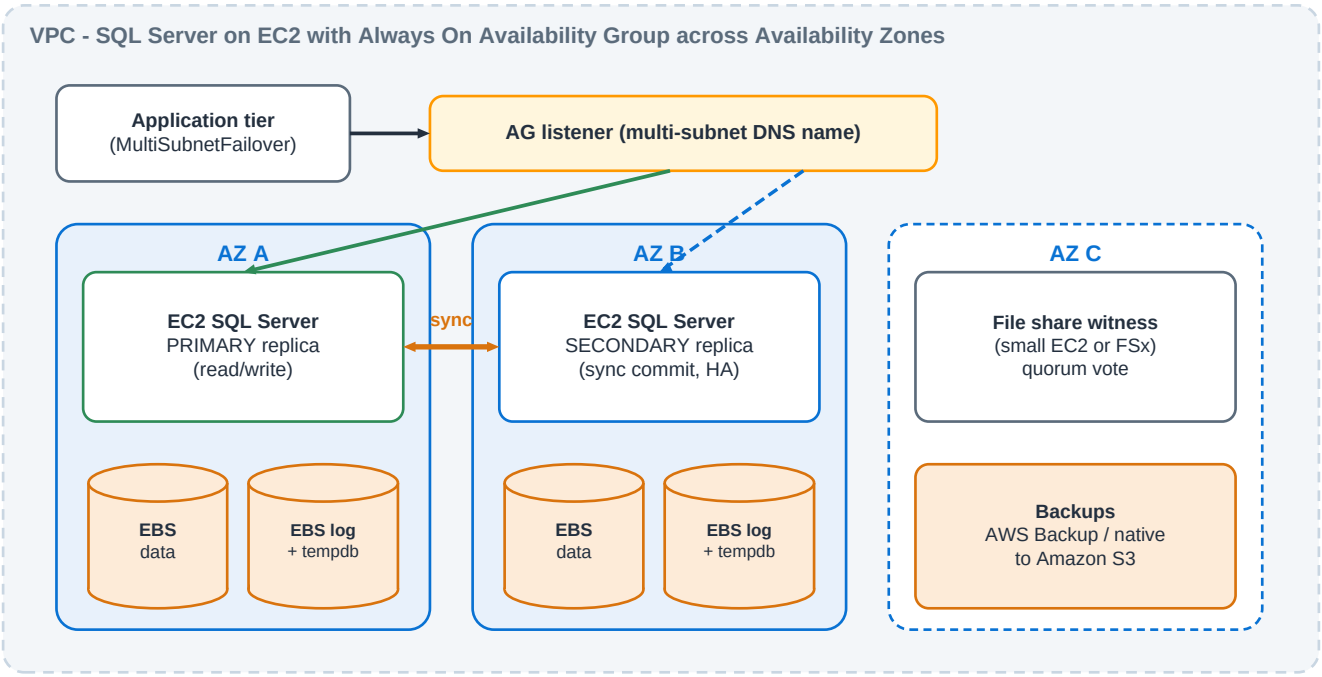


Figure 10.1 - SQL Server on EC2: Always On AG across Availability Zones

Area	Design guidance
Instance	EBS-optimised Nitro instance (memory-optimised R-family or storage-optimised for heavy I/O); size from RDS CloudWatch history
Operating system and SQL	Windows Server with SQL Server (AWS licence-included AMIs, or BYOL media); keep patch levels aligned with testing
Storage layout	Separate volumes for data, log, tempdb (or local NVMe for tempdb) and backups; format with 64 KB allocation unit; gp3 or io2 Block Express
HA	Always On AG with synchronous commit across AZs (Enterprise for advanced features; Standard supports basic AGs), WSFC, multi-subnet listener, file share witness (FSx for Windows or small EC2); or an FCI with FSx for Windows File Server
Active Directory	Domain-joined nodes (AWS Managed Microsoft AD or self-managed AD) for WSFC and Windows authentication
Backups	Native backups to S3 (maintenance scripts), AWS Backup with VSS-aware EBS snapshots, cross-Region copies for DR
Patching and ops	AWS Systems Manager Patch Manager, State Manager, Session Manager; CloudWatch agent for OS and SQL counters
Security	Private subnets, security groups, EBS encryption with KMS, TDE if required, hardened OS baseline, endpoint protection
Automation	CloudFormation/Terraform; AWS Launch Wizard for SQL Server can deploy HA reference architectures

10.3 What you must now build yourself

On RDS this was automatic...	On EC2 you must...
Automated backups and point-in-time restore	Schedule full/diff/log backups, store them in S3, test restore, manage retention
Multi-AZ failover	Build WSFC + AG (or FCI), tune quorum and listener, test failovers
OS and SQL patching in a window	Run patching with Systems Manager; coordinate AG patch order
Metrics, Performance Insights and alarms	Install the CloudWatch agent, create dashboards and alarms, collect wait stats
Storage autoscaling	Expand EBS volumes and Windows partitions manually or with automation
Encryption and certificates	Manage TDE certificates, TLS certificates and key rotation
Event notifications	Wire SQL Agent alerts, CloudWatch and EventBridge to SNS
Minor version upgrades	Plan, test and apply cumulative updates yourself

10.4 Migration methods from RDS

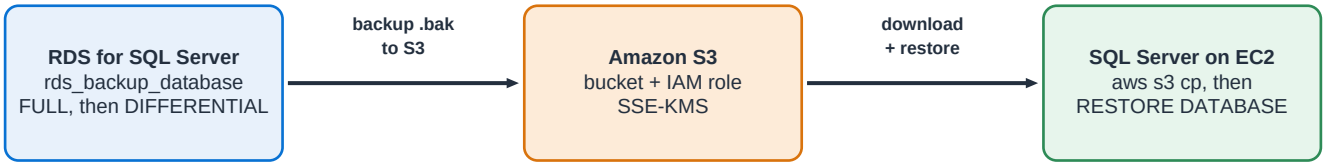
RDS exposes native **full and differential** backups to S3 but **no transaction log backups**. That shapes the options:

Method	Downtime	Best for	Notes
Native full + differential backup via S3	Backup + copy + restore of the final differential	Most databases; simple and faithful	Take the full backup early; take the differential in the cutover window. Last native full is the differential base.
AWS DMS (full load + CDC)	Minutes	Large DBs, minimal downtime	Enable CDC on RDS databases (rds_cdc_enable_db); pre-create schema on EC2; no logins/jobs
Transactional replication	Minutes	Object-level control, staged moves	Initialise EC2 subscriber from a native backup, then replicate; confirm RDS publisher/distributor support for your version
BACPAC / SqlPackage, bcp, SSIS	Long	Small DBs, table moves	Portable but slow for large data

You cannot attach an RDS snapshot or EBS volume to your own EC2 instance. RDS storage is not exposed; use native backups, DMS or replication.

10.5 Step-by-step: native backup path

Path A - Native backup (full + differential) via S3



Path B - AWS DMS or replication (low downtime)

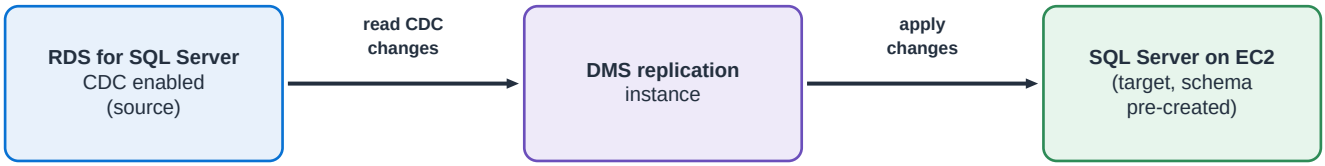


Figure 10.2 - Two paths from RDS to SQL Server on EC2

Step 1: Prepare the EC2 target

- Build the EC2 SQL Server with a **version equal to or newer** than the RDS version and an edition that supports every feature used.
- Match **collation** and set sensible file paths. Attach an **IAM instance profile** that can read the migration bucket; install the AWS CLI.
- Ensure the RDS instance has the **SQLSERVER_BACKUP_RESTORE** option and an IAM role that can write to the bucket (same Region).

Step 2: Back up from RDS to S3

```
EXEC msdb.dbo.rds_backup_database
     @source_db_name = 'SalesDB',
     @s3_arn_to_backup_to = 'arn:aws:s3:::my-migration-bucket/export/SalesDB_full.bak',
     @overwrite_s3_backup_file = 1,
     @type = 'FULL';

EXEC msdb.dbo.rds_task_status @db_name = 'SalesDB';   -- wait for SUCCESS
```

Step 3: Download and restore on EC2 (WITH NORECOVERY)

```
aws s3 cp s3://my-migration-bucket/export/SalesDB_full.bak E:\Restore\

-- On the EC2 SQL Server: find logical names, then restore and keep it restoring
RESTORE FILELISTONLY FROM DISK = 'E:\Restore\SalesDB_full.bak';
RESTORE DATABASE SalesDB FROM DISK = 'E:\Restore\SalesDB_full.bak'
  WITH MOVE 'SalesDB'     TO 'F:\SQLData\SalesDB.mdf',
       MOVE 'SalesDB_log' TO 'G:\SQLLog\SalesDB_log.ldf',
       NORECOVERY, STATS = 10;
```

Step 4: Cutover with a differential backup

```
-- On RDS, after stopping the application (or setting the DB READ_ONLY)
EXEC msdb.dbo.rds_backup_database @source_db_name = 'SalesDB',
     @s3_arn_to_backup_to = 'arn:aws:s3:::my-migration-bucket/export/SalesDB_diff.bak',
     @overwrite_s3_backup_file = 1, @type = 'DIFFERENTIAL';

-- On EC2: copy the file and bring the database online
RESTORE DATABASE SalesDB FROM DISK = 'E:\Restore\SalesDB_diff.bak' WITH RECOVERY, STATS = 10;
```

Step 5: Finish

- If TDE is enabled on RDS, back up the **TDE certificate** with the RDS procedures and restore it on EC2 *before* restoring the database (or remove TDE before the backup).
- Create logins with matching SIDs, fix orphaned users, re-create SQL Agent jobs, linked servers and Database Mail, and set server options (MAXDOP, memory) that RDS parameter groups used to hold.
- Raise compatibility level only after testing; update statistics; run `DBCC CHECKDB`.
- Repoint the application (DNS or connection strings). Keep the RDS instance read-only as a fallback, then take a final snapshot and delete it once the rollback window closes.
- Configure the EC2 backups, AG, patching and monitoring **before** go-live; these are no longer automatic.

> **Plan for rollback ('fall forward')**
> AWS describes a fall-forward pattern: after moving in the opposite direction, you keep a copy of the data in sync so you can switch back. For RDS to EC2, keep the RDS instance intact and read-only for an agreed period; if the EC2 platform misbehaves, you can restart from the last consistent state with minimal data loss, but any writes made on EC2 after cutover must be moved back explicitly (DMS or backups).

10.6 Cutover runbook (RDS to EC2)

When	Action	Owner
T - 2 weeks	Rehearse the full path with production-size data and the real network; record each step's duration	DBA
T - 1 day	Take and restore the FULL backup early; verify EC2 is in NORECOVERY and healthy	DBA
T - 30 min	Pause jobs and schedulers; confirm backups and rollback steps	DBA / App
T - 15 min	Stop application writes	App team
T	Differential backup on RDS, copy, restore WITH RECOVERY on EC2	DBA
T + 20 min	Create/verify logins, fix orphaned users; enable jobs; repoint connections	DBA / App
T + 40 min	Smoke tests, performance and log checks	Everyone
T + 2 hours	Go/no-go; if no-go, resume on RDS (read-write) and investigate	Project lead
T + 2 weeks	Final RDS snapshot, delete RDS instance, review cost and operations	DBA / Finance

Appendix A: DBA Cheat Sheet

RDS stored procedures commonly used

Procedure / command	Purpose
EXEC msdb.dbo.rds_get_high_availability_option	Shows Single-AZ, AlwaysOn, Mirroring or Block-level replication
msdb.dbo.rds_backup_database	Native backup to S3 (FULL or DIFFERENTIAL)
msdb.dbo.rds_restore_database	Native restore from S3 (FULL/DIFFERENTIAL, NORECOVERY option)
msdb.dbo.rds_restore_log	Restore a transaction log backup from S3
msdb.dbo.rds_finish_restore	Finish a restore left in RESTORING state
msdb.dbo.rds_task_status / rds_cancel_task	Track or cancel backup/restore tasks
rdsadmin.dbo.rds_read_error_log	Read the SQL Server error log
msdb.dbo.rds_cdc_enable_db	Enable change data capture on a database (for DMS and CDC consumers)
rdsadmin.dbo.rds_modify_db_name	Rename a database (not allowed while Multi-AZ is on)

Names and parameters can differ by version; confirm in the RDS documentation 'Common DBA tasks for Microsoft SQL Server'.

AWS CLI commands commonly used

Task	Command
Describe the instance	aws rds describe-db-instances --db-instance-identifier sql-prod-01
Resize or change settings	aws rds modify-db-instance --db-instance-identifier sql-prod-01 --db-instance-class db.m6i.4xlarge --apply-immediately
Make Multi-AZ	aws rds modify-db-instance --db-instance-identifier sql-prod-01 --multi-az --apply-immediately
Test failover	aws rds reboot-db-instance --db-instance-identifier sql-prod-01 --force-failover
Manual snapshot	aws rds create-db-snapshot --db-instance-identifier sql-prod-01 --db-snapshot-identifier before-release
Copy snapshot to another Region	aws rds copy-db-snapshot --source-db-snapshot-identifier arn:... --target-db-snapshot-identifier x --region ap-southeast-1 --kms-key-id alias/...
Point-in-time restore	aws rds restore-db-instance-to-point-in-time --source-db-instance-identifier ... --target-db-instance-identifier ... --use-latest-restorable-time
Read replica / promote	aws rds create-db-instance-read-replica ... | aws rds promote-read-replica ...
Show recent events	aws rds describe-events --source-type db-instance --duration 1440
Download a log file	aws rds download-db-log-file-portion --db-instance-identifier sql-prod-01 --log-file-name log/ERROR

Appendix B: Which Hosting Model and Migration Method?

Situation	Choose
Standard SQL Server workload, want minimal admin	Amazon RDS for SQL Server (Multi-AZ for production)
Need OS access but want managed automation	RDS Custom for SQL Server
Need unsupported feature, full control or BYOL media	SQL Server on EC2
On-prem to RDS, DB under ~500 GB, hours of downtime OK	Native full backup/restore
On-prem to RDS, large DB, minutes of downtime	Native full + differential + log restore (or DMS)
On-prem to RDS, only some tables or continuous sync	AWS DMS full load + CDC, or transactional replication
Tens of TB, poor bandwidth	Snowball Edge for the seed, then logs or CDC online
RDS to EC2, hours OK	Native full + differential via S3
RDS to EC2, minutes of downtime	DMS or transactional replication

Appendix C: Glossary

Term	Meaning
AZ	Availability Zone: one or more data centres in a Region with independent power and networking
AG / DAG	Always On Availability Group / Distributed Availability Group
CDC	Change Data Capture - records inserted/updated/deleted rows for downstream replication
DBM	Database Mirroring - older SQL Server HA technology
DMS	AWS Database Migration Service
EBS	Elastic Block Store - network block storage used by RDS and EC2
FCI	Failover Cluster Instance - shared-storage clustering for SQL Server
IAM	Identity and Access Management
KMS	Key Management Service for encryption keys
Multi-AZ	RDS deployment with a synchronous standby in another AZ
Option group / Parameter group	RDS containers for optional features / engine settings
PITR	Point-in-time restore
RDS Custom	RDS flavour that allows OS and database customisation
RPO / RTO	Recovery point / time objective
S3	Simple Storage Service - object storage
Security group	Stateful virtual firewall for AWS resources
VPC	Virtual Private Cloud - your private network in AWS
WSFC	Windows Server Failover Clustering, required by AGs and FCIs on EC2

Appendix D: Where to Learn More

RDS for SQL Server changes quickly: new versions, limits and features appear several times a year. Treat this guide as an architecture and process reference and confirm exact numbers in the AWS documentation.

Topic	Where to look (docs.aws.amazon.com and aws.amazon.com/blogs/database)
Everything RDS for SQL Server	Amazon RDS User Guide: 'Amazon RDS for Microsoft SQL Server'
Versions and support dates	'Microsoft SQL Server versions on Amazon RDS' and 'Version policy'
Multi-AZ details	'Multi-AZ deployments for Amazon RDS for Microsoft SQL Server'
Read replicas	'Working with read replicas for Microsoft SQL Server in Amazon RDS'
Native backup and restore	'Importing and exporting SQL Server databases using native backup and restore'
Migration guidance	AWS Prescriptive Guidance: 'Migrating Microsoft SQL Server databases to the AWS Cloud'; AWS Database Migration Guide
Self-managed on EC2	AWS Prescriptive Guidance: SQL Server on Amazon EC2 best practices and HA/DR
RDS Custom	'Working with Amazon RDS Custom for SQL Server'
Pricing and SLAs	AWS pricing calculator and the Amazon RDS Service Level Agreement